

MR Technical Manual

A technical history of boundaries, speed, and extensibility

Dr. Michael 'iDoc' Raus

5 August 2026

Preface

This is neither a class catalogue nor retrospective self-congratulation. It explains why MR is built differently from a conventional terminal editor in several important respects, which experiments were abandoned, and which limits remain despite the successes. Its sources are the repository, architecture contracts, planning documents, Kanban entries, and version history. Where those sources do not support a claim, this manual does not make it.

The MRMAC Reference Manual documents the language and its primitives. This book tells a different story: how document handling, runtime, UI, and user customisation became one system whose parts do not quietly take ownership of one another.

Contents

Preface	ii
I Why MR exists	1
1 Not another editor	2
2 Standing on the shoulders of others	3
II The document as workspace	4
3 A gigabyte file in five milliseconds	5
3.1 Finite packets instead of deferred serial work	5
3.2 Retained information and render consumers	6
3.3 Moving windows, checkpoints, and ledgers	7
3.4 Line-break scan primitives	7
4 Derived work must not stop the editor	9
4.1 Task envelope and scheduling boundary	9
4.2 Lanes classify work; owners define independence	9
4.3 External sources: the lifetime-bound exception	10
4.4 Worker completion and the result mailbox	10
4.5 Cancellation and observability	11
4.6 Two observation scales	11
5 A minimap with eight dots per character	14
5.1 Geometry and the sampling window	14
5.2 Worker-side Braille reduction	15
5.3 Result adoption and cache continuity	15
5.4 Bounded reuse and controlled churn	15
5.5 Overlay masks and TVision projection	15
6 A block is geometry, not a clipboard string	18
6.1 Two coordinate systems and one explicit lifecycle	18
6.2 Typed transfer, staged mutation, and the system clipboard	18
III A coordinated workspace	20
7 Bento: one task, several views	21
7.1 Dirty projection without a second UI model	21
7.2 File Compare as an ordered finite pipeline	23
IV MRMAC as an integrated runtime	25
8 A language that belongs to the editor	26
8.1 One compiler, three deliberate products	26
8.2 Hash handles, chained lvalues, and scope ownership	26

9	Inside the MRMAC virtual machine	28
9.1	The bytecode image and execution loop	28
9.2	One value model, several lifetimes	28
9.3	Opcodes end at typed application boundaries	29
9.4	The execution profile chooses the legal route	29
9.5	Yielding, debugging, cancellation, and completion	30
9.6	Bytecode calls and macro calls are different operations	30
9.7	Authority, persistence, and failure containment	30
9.8	Why this is a VM rather than a callback catalogue	31
9.9	Observability without a shadow runtime	31
10	From a broken macro clock to Execution Sessions	34
10.1	Closures, timers, and explainable skips	34
11	Coprocessor, staging, and the UI thread	36
12	MMP: modeless programs without a second UI toolkit	37
12.1	One value model, several projections	37
12.2	Callbacks are sessions, not event-handler re-entry	37
13	A debugger for the complete runtime	39
13.1	Source identity and source maps	39
13.2	Natural execution routes and non-blocking control	39
13.3	Cold configuration and regenerated truth	39
13.4	Scopes and complete mutable values	40
V	The editor as a personal environment	42
14	settings.mrmac: canonical bootstrap and executable state	43
15	Keymaps as a programmable input language	45
16	Profiles and automatic setup	46
VI	Syntax as a shared core	47
17	From an external generator to an integrated syntax model	48
18	Trainers: the workbench for language features	49
VII	Spaces and the Unix world	50
19	Virtual desktops and resumable workspaces	51
20	Git as a file-scoped external view	52
21	MFS and Acquire: Unix output as working material	53
VIII	Decisions that left traces	54
22	The Hex restart and the LSP train	55
23	Measured limits, not wishful thinking	56
	Outlook	57

Part I

Why MR exists

Chapter 1

Not another editor

MR began with the intention of carrying the character of Multi-Edit forward, not as a nostalgic surface but as a way of working. An editor should be immediate, keyboard-capable, and at home in a terminal. At the same time, it should not treat large files, many windows, build output, and automation as foreign bodies.

This heritage produced several non-negotiable goals: speed, a practically unbounded workspace, reliable editing operations, extensibility and customisation by the user, and responsiveness under load. The close relationship between MRMAC and the compiled application is therefore not an item at the end of a plugin list. It is part of the product's definition. Decoupling matters just as much: a document may be edited while derived work is being produced, but a background result must not write itself into a screen that has moved on in the meantime.

The guiding principle is consequently not *as many features as possible*, but *as much productive power as possible without sacrificing control*. Each chapter asks a concrete question: which technical problem threatened that goal, what idea shaped the solution, and where did MR deliberately reject a seemingly convenient alternative?

Chapter 2

Standing on the shoulders of others

MR does not attempt to carve every wheel from scratch. Its distinctive character comes precisely from the boundary between inherited foundations and its own semantics.

The working model comes from Multi-Edit and the MEMAC tradition: windows, blocks, macros, and personal interaction should be ordinary parts of editing rather than exceptional add-ons. MRMAC preserves that lineage, while being its own compiled, in-memory runtime.

Magiblot's TVision provides the terminal-native object world. TVision owns focus, Z order, event routing, view lifetime, dialogs, and drawing dispatch. MR extends that world through its intended mechanisms instead of placing a second screen model or event loop beside it. The dependency is maintained as an upstream subtree with a small patch queue. This is a deliberate relationship: upstream remains identifiable and local changes remain reviewable.

The frequently proposed alternative is not really TVision *or* `ncursesw`. MR links against `ncursesw`; it is part of the terminal platform beneath TVision. The actual choice was TVision rather than building a window, focus, dialog, and drawing system directly on bare curses. For an editor with virtual desktops and a long Multi-Edit UI tradition, that was a useful trade: less invented infrastructure, more energy for documents and runtime behaviour.

PCRE2 supplies Perl-compatible regular-expression semantics for search and multi-file work. POSIX supplies, among other things, `mmap`, processes, and pipes. Pango and Cairo support PDF export. These components are not hidden MR subsystems. Each solves a clearly bounded problem; responsibility for placing its results in the editor model remains with MR.

Part II

The document as workspace

Chapter 3

A gigabyte file in five milliseconds

The striking number needs to be told accurately. The `enwik9` file used in the repository contains exactly 1,000,000,000 bytes and 13,147,025 lines. On the measured development machine, MR makes this document available as an editable document in about five milliseconds.

That does not mean that a gigabyte is copied from storage and fully analysed in five milliseconds. The number is the measured *mapped load* time. MR opens a regular file as a private, read-only memory mapping and builds its logical document state on top of that mapping. The operating system brings pages in on demand; the editor does not first copy the original into a mutable string.

A Piece Table sits above the mapping. The original remains the source of pieces, new input goes into the add buffer, and the logical order is expressed as references to those sources. An edit therefore does not move a gigabyte of text. It changes piece boundaries and adds new data to the add buffer. This model serves more than load time: it makes editing, snapshots, undo boundaries, and later derived work variations of the same data economy.

Figure 3.1 on page 6 makes the ownership boundary explicit. A background task receives a versioned read snapshot, never the mutable live object. It may return a derived result or a staged edit, but it cannot adopt that work itself.

The line index then warms independently from interaction. Until an exact total is available, the editor uses a bounded estimate and local cursor-relative navigation. The `enwik9` milestone was therefore not satisfied by a fast “opened” message. It required immediate restore and drawing, responsive scrolling and large jumps, and eventual exact metrics without a hidden UI-thread full scan.

3.1 Finite packets instead of deferred serial work

The general task envelope, one-shot worker model, and adoption path are described in Chapter 4. For large documents, the relevant domain fact is that line index, display width, syntax, folding, and minimap are separate pipelines. Their finite packets carry a concrete execution owner and a versioned `ReadSnapshot`; their domain ledgers, ordering rules, and retained results are deliberately different. Focus loss is not a pause signal.

Line-index packets are arithmetic byte windows of eight MiB. The UI does not search for line boundaries before submission. A document-bound ledger records the contiguous prefix confirmed from BOF, out-of-order packet results, and active reservations. Planning covers gaps only, so a cursor jump may start new BOF- and EOF-directed work without duplicating an older packet. Sparse line checkpoints are retained every 4,096 line breaks; the exact total becomes authoritative only when the confirmed prefix reaches EOF. Goto-line and cold large-file drawing consult confirmed checkpoints or wait for ledger progress; they no longer count linearly from BOF on the UI thread.

Display-width work begins only after that exact line index exists. A generation splits the indexed lines into at most one packet per allowed core. Each worker addresses its first line once and then advances linearly with `nextLine`; it does not repeat `lineStartByIndex` for every line. The result is one maximum terminal-cell width per packet, not a cache of all line widths. Syntax, folding, minimap, indentation, and rendering consume the common line-addressing basis, not the display-width result.

Syntax and folding have a stricter adoption rule because their state moves forward through the document. Text acquisition may run in parallel, but a packet becomes visible only from a confirmed `stateIn`. A speculative packet whose entry state differs is discarded and only that finite packet is recomputed from the confirmed predecessor. Syntax publishes sparse state checkpoints and only the token runs required for the viewport and prefetch range. Folding similarly publishes a checkpoint containing its structural stack and bounded language look-back. Neither pipeline invents a closing state at a packet boundary.

PIECE TABLE OWNERSHIP AND SNAPSHOT BOUNDARY

Original bytes remain immutable; edits change references and append data; background work receives a version ticket.

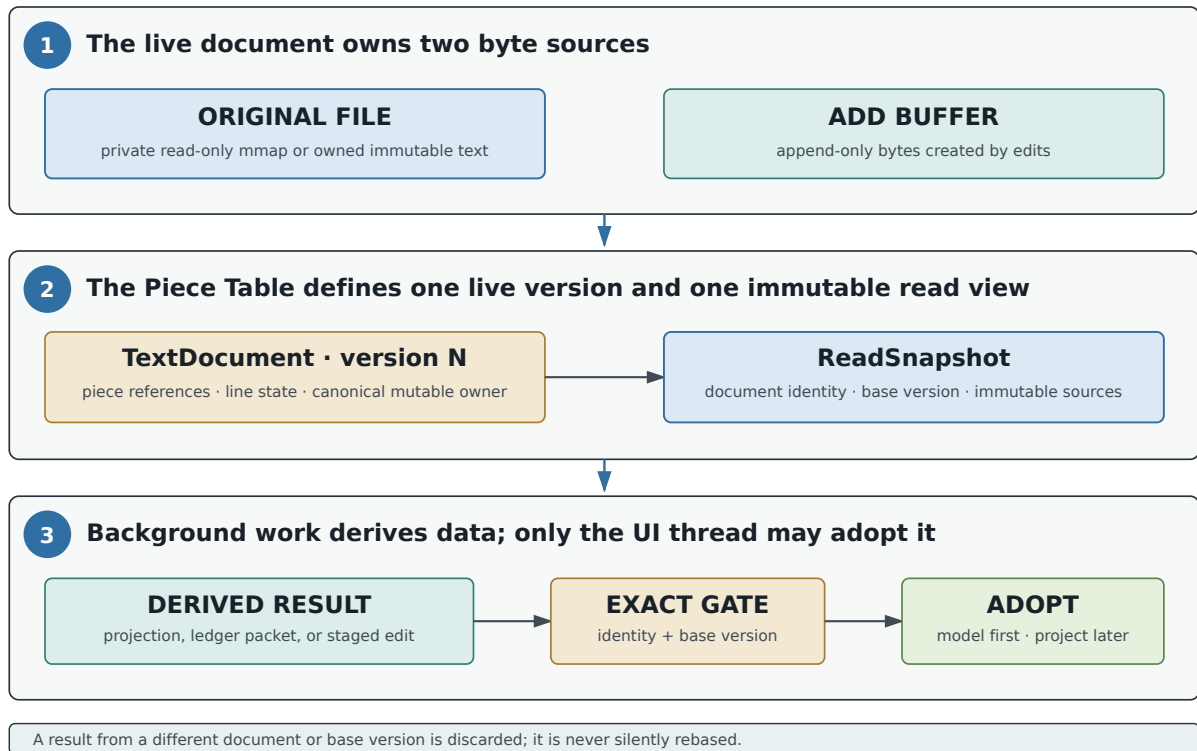


Figure 3.1: Piece Table ownership and the snapshot boundary. The mapped or owned original and append-only buffer are the sources of the live document. Background work receives a versioned `ReadSnapshot`; only the UI may adopt a result whose document identity and base version still match.

The minimap likewise uses finite text and overlay projection packets. Its editor-local windowing, reuse, and rendering rules belong to Chapter 5; here it matters that the minimap consumes the common line-addressing basis without becoming another line or syntax index.

Figure 3.2 on page 7 shows the four derived projections at one viewport position together with their task telemetry. The numbered callouts identify rendered consumers and instrumentation, not additional scan stages.

3.2 Retained information and render consumers

The payloads and caches preserve only the information required by their consumers:

Projection	Retained information	Direct consumer
Line index	confirmed BOF prefix, sparse byte/line checkpoints, out-of-order packets, active reservations and exact total at EOF	line numbers, byte-to-line and line-to-byte navigation, scrollbar and document metrics
Display width	generation packets and one maximum terminal-cell width per packet	horizontal viewport extent and scrollbar limit
Syntax	line start, <code>stateIn</code> , token runs with column, length and token class, and <code>stateOut</code>	coloured editor text and the state checkpoint chain for subsequent lines
Folding and outline	confirmed structural checkpoints, validated segment spans, nesting and gutter projection, plus bounded source acquisition	fold gutter, closed-line projection and partial or complete outline generation
Minimap	bounded warm-window ledger, immutable Braille row patterns, document-line intervals and revisioned overlay components	minimap cells, viewport marker and line-based overlays
Performance view	worker ordinal, owner, core, lane, lifecycle state, generation, direction, span and queue/run/accept/adopt timing	observability only; never a semantic input to an editor projection

The coprocessor adoption contract in Chapter 4 applies to every row. The table names the extra domain state that a result must satisfy before its direct consumer may use it.

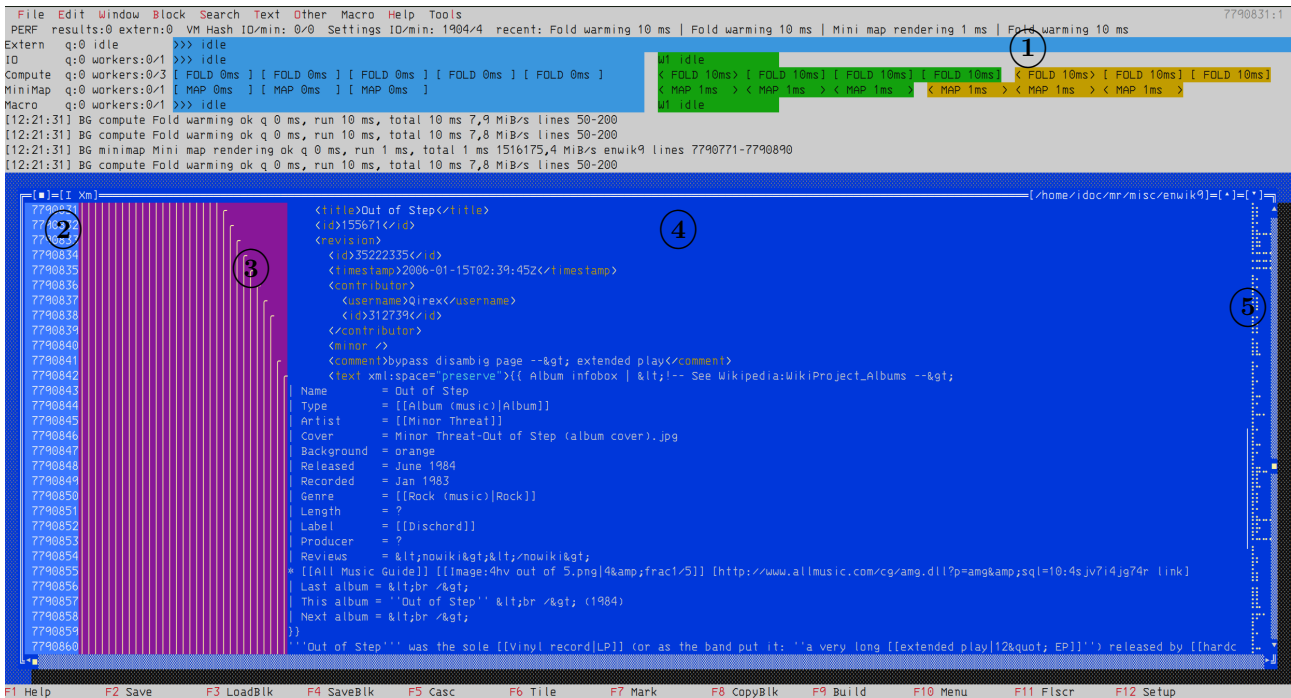


Figure 3.2: Deferred projections while viewing enwik9 near line 7,790,831. The uniformly annotated plate identifies (1) lifecycle and packet telemetry, (2) adopted line-number projection, (3) validated fold structure, (4) syntax tokens published from a confirmed state chain, and (5) the viewport-anchored minimap.

3.3 Moving windows, checkpoints, and ledgers

Figure 3.3 on page 8 shows the common shape without erasing the domain differences. A viewport demand creates a generation and reserves only uncovered packet spans. Finite workers may complete out of order and may remain useful after a cursor jump. The ledger first accepts matching results; the domain then advances its confirmed frontier or visible window as its ordering rule allows.

For syntax, BOF direction means context acquisition before the viewport, never backward lexical evaluation. A confirmed line state may serve directly as the next anchor; otherwise sparse checkpoints lead back towards BOF. Packet count is additionally limited by a minimum useful grain of sixteen lines. For folding, fixed 65,536-line acquisitions populate a document-wide canonical context chain. Structural validation advances in checkpoint order, while the visible segment projection remains immutable and searchable by segment boundaries. Cursor movement within the same document version does not rebuild that canonical chain.

For the minimap the ordering constraint is weaker. Its acquisition window is directionally biased and covers at least four times the allowed-core count in visible minimap windows. At most one packet per allowed core is materialised, and the ledger retains a bounded number of complete windows. Small movement inside the warmed region therefore creates no new worker wave. An edit, language change, geometry change, or incompatible document version invalidates only the state whose key includes that dimension.

3.4 Line-break scan primitives

Line navigation, line numbering, packet acquisition, and Piece Table scans require the same three operations: locate the next line break, locate the previous line break, and count line breaks in a byte range. These operations are implemented in the line-index core. On x86 builds with SSE2, the core loads sixteen bytes at a time, compares them with carriage return and line feed, and converts the resulting bit masks into positions and counts. A scalar implementation is the fallback for platforms without the vector path.

The scanner carries CRLF state across vector blocks and handles the scalar tail, preserving the same line model at vector and chunk boundaries. The counting primitive operates on direct mapped text and on individual Piece Table chunks; the direct path also uses vector search in both directions. Packet boundaries preserve CRLF semantics, and a sequential line range is addressed once before advancing locally. All derived pipelines therefore consume one line model rather than maintaining feature-specific text indexes.

FINITE PACKETS, OWNER LEDGERS, AND ORDERED PUBLICATION

A lane classifies work; the domain owner reserves spans and decides when an exact result becomes visible.

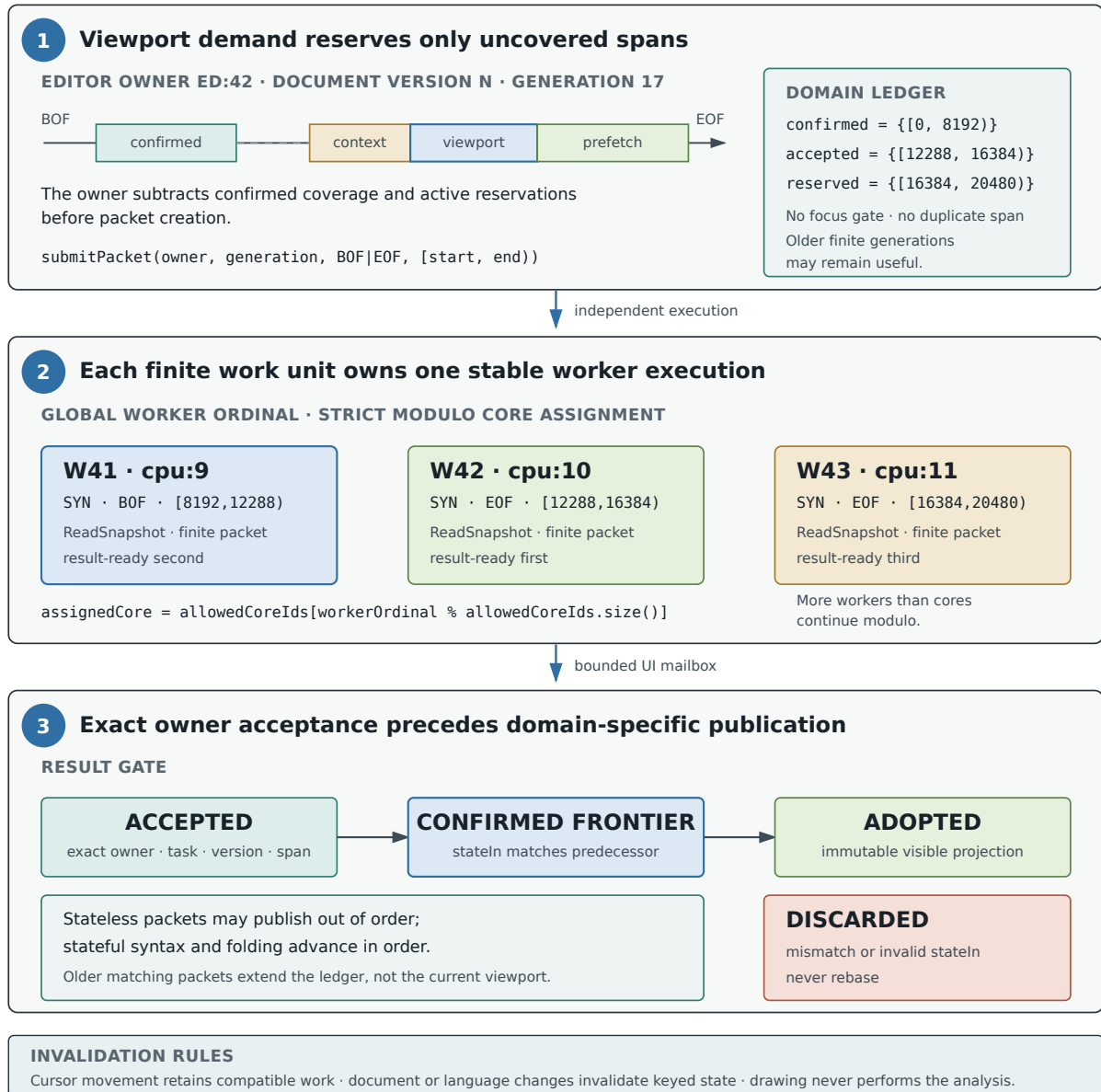


Figure 3.3: Finite packet planning and ordered publication. The viewport demand reserves uncovered spans; independently pinned workers return immutable packet results; the owner ledger accepts exact matches and publishes only a confirmed state chain or complete visible window. Older finite packets may still enlarge the ledger without replacing a newer viewport projection.

Chapter 4

Derived work must not stop the editor

The coprocessor is a lifecycle and delivery boundary, not a fixed worker pool and not a general executor from which a worker may call back into the application. Four finite-work lanes classify I/O, compute, minimap, and macro work for telemetry and dispatch. Registered external sources use a separate lifetime-bound route. Neither mechanism serializes independent owners. Figure 4.1 on page 12 shows the complete path from a domain packet to an adopted result.

4.1 Task envelope and scheduling boundary

`submit` and `submitPacket` construct a `TaskInfo` with a unique task identifier, lane and task kind, execution-owner kind and owner-local identifier, document identity and base version, cancellation state, and—for packet work—generation, direction and half-open span. Document-derived submitters capture the coherent read snapshot or immutable input on which the task operates. The document/version pair is an adoption ticket, not permission for the worker to inspect a live view.

The old one-shot coalescing API was removed because every invocation created a new execution state and therefore had no shared queue in which to find an older key. Coalescing now belongs to the owning domain: a pane may retain one active task and one newest requested successor, while a ledger may reserve already covered spans. This keeps scheduler mechanics separate from semantic equivalence.

Every ordinary finite submission creates its own `std::thread`. The worker receives the next monotone `workerOrdinal`; its assigned core is computed once as

```
allowedCoreIds[workerOrdinal % allowedCoreIds.size()]
```

and remains stable for that worker's lifetime. The number of workers is not capped by the number of cores. More workers continue around the same modulo sequence, so independent work does not become a lane-wide queue merely because all cores already have an assignment.

4.2 Lanes classify work; owners define independence

Exactly four ordinary lanes appear in the coprocessor snapshot and Performance Panel: `Io`, `Compute`, `MiniMap`, and `Macro`. Four is not a CPU topology, a concurrency limit, or a priority ladder. The number follows from four observably different finite-work contracts. Collapsing them would hide why work exists and which result path must validate it; adding a lane for every feature would merely duplicate task kind and owner identity.

I/O. The `Io` lane classifies finite command, Git, process, and similar operating-system work whose dominant uncertainty is external latency. Its normal result describes command completion, output, or failure rather than a document-derived projection. The classification keeps such latency and error behaviour visible without granting a finite command the lifetime semantics of a registered stream.

Compute. The `Compute` lane carries deterministic derived work over immutable input: line-index and display-width packets, syntax and folding, Bento diagnostics and outlines, HexBento projections, and File Compare stages. These results are normally accepted against document identity, version, generation, span, pane role, or another domain checkpoint. Bento is therefore a task kind and pane owner on `Compute`, not a fifth lane.

MiniMap. The **MiniMap** lane identifies per-editor text-density and overlay projection packets. They are computational, but their demand is driven by viewport movement, geometry, visible row windows, and rapidly changing overlay state. Keeping this class separate makes minimap churn, coverage, and adoption visible independently of canonical line, syntax, and fold preparation. It does not give the minimap a privileged worker or a private result mailbox.

Macro. The **Macro** lane identifies finite **MacroJob** executions. Their owner is an Execution Session, and their results differ from document projections by carrying macro log, session, or staged-effect state. VM profiling and session routing are described in Chapters 9 and 10. A paused macro holds its VM/session state but no lane worker.

The distinction is thus semantic and operational. The lane name gives telemetry and dispatch the first workload category; **TaskKind** identifies the concrete operation; execution-owner kind and owner-local ID name whose work it is; document/version and generation fields decide whether a result may still be used. No one of those axes can replace the other three. None of the four lane names denotes a shared single-worker FIFO.

An editor window, Bento pane, Execution Session, external source, or channel is an execution owner. Several packets may belong to one owner, and the same document may be viewed through several independent owners. Result routing uses the exact owner and task identity; two windows over the same document cannot clear or adopt one another's work.

The deliberate exception is a technically blocking source whose lifetime consists of `poll()`, `read()`, process wait, or an equivalent external wait. The implementation tags this route as `Lane::Extern`, but does not include it among the four finite lane snapshots. Registered sources have their own source snapshot and lifetime because they are not one-shot work. A registered file, journal, device, network, or pipe source may keep one exclusive worker until EOF, close, cancellation, error, or process termination. It never carries another owner's work. Journal cancellation closes the local read side, terminates and reaps the process group with bounded escalation, and publishes the exact viewer target before the worker is joined. This exception does not relax exact ownership or UI-thread adoption.

An MRMAC Execution Session is not such a source merely because it is asynchronous. A finite VM computation is a **MacroJob**; a session paused for UI adoption holds logical execution state but no blocked worker. If that session owns a real external process or terminal source, source and session lifetimes remain separate.

4.3 External sources: the lifetime-bound exception

A registered file, journal, device, network, or pipe source receives one source ID and one exclusive worker. That worker may remain alive until EOF, close, cancellation, error, or process termination because waiting is the work it owns. It may not execute a second source's request. Data still crosses the ordinary result boundary as typed chunks, each carrying the source and exact target buffer. The UI dispatcher appends or prepends the chunk only if that target still exists; closing one viewer cannot redirect its late output into another communication window.

The file-tail route records device, inode, and byte offset. It reads new data with bounded `pread` calls and reopens the path after replacement or truncation. Optional timestamps are added at logical line starts. Search matches are projected against the current search expression during adoption, so highlight ranges, message-line reporting, terminal bell, and configured audio notification describe the text actually entering the live view rather than an obsolete worker-side UI state.

The journal route owns a different blocking boundary. A child process runs `journalctl -f` in its own process group and writes to a non-blocking pipe. The source worker alternates bounded `poll`, pipe reads, and process-state checks. Cancellation first closes the local read side and sends `SIGTERM` to the process group. If the group does not finish within the bounded poll sequence, cancellation escalates to `SIGKILL`; the child is reaped before the worker becomes finished. This prevents a closed journal window from leaving either a blocked reader or descendants behind.

Figure 4.2 on page 13 separates the two acquisition routes from their common chunk-adoption and cancellation contract.

4.4 Worker completion and the result mailbox

The one-shot worker publishes creation, assignment, queue, running, and result-ready transitions, performs its finite task outside scheduler locks, and appends the immutable result to the mutex-protected central FIFO. Exceptions become failed results; cancellation remains cooperative and is checked at bounded points. No lane has a private path to the screen.

`pump(maxResults)` drains at most the requested count—eight by default—whereas `pumpFor` stops after the supplied time budget once a result has been handled. Both invoke the installed result handler on the UI thread.

The central `handleCoprocessorResult` performs payload dispatch; it does not replace the owning domain's validation.

Dispatch locates the exact task-owning editor, pane, session, source, or channel. The domain checks owner, task, generation, document and version, plus its own geometry, language, checkpoint, or conflict conditions. A stateful pipeline may first mark a result `accepted` in its ledger and later `adopted` into the visible model. A mismatch produces `discarded`; it is never rebased implicitly.

4.5 Cancellation and observability

`cancelTask` sets the shared cancellation state; a worker is not killed at an arbitrary instruction. Worker lifetime uses explicit `std::thread`, a directly visible atomic stop state, `condition_variable`, and `join`. Retirement joins only workers already marked finished, so normal UI pumping does not wait for running work. Synchronous join remains the unavoidable shutdown lifetime barrier.

The performance snapshot separates identities from measurements. It reports the execution owner, task, `workerOrdinal`, OS thread identifier and assigned core as distinct fields. Its bounded lifecycle ring records `created`, `assigned`, `queued`, `running`, `result-ready`, `accepted`, `adopted`, `discarded`, `stopping`, and `finished`. Queue, run, accept, adopt and total durations, packet generation, direction and span are observability outputs only; they never become semantic inputs to a projection.

4.6 Two observation scales

The Performance Panel is the global diagnostic projection of this telemetry. It combines the current coprocessor snapshot, the bounded lifecycle event ring, one-second activity counters, lane and owner totals, affinity failures, and selected VM-hash and settings I/O rates. A refresh signature avoids drawing an unchanged panel; running work adds a quarter-second time component so durations remain live. The panel neither owns nor cancels any task.

The editor-frame activity marker is deliberately narrower. It projects only work attributable to that window: explicitly tracked external commands and macro jobs together with pending line-index, display-width, syntax, folding, and minimap packets. Its provider constructs human-readable rows from the owner's current state. Hover opens a transient TVision window, click pins it, and Escape or close dismisses it. No overlay becomes a second scheduler and no popup retains a worker handle after its owner has stopped reporting it.

COPROCESSOR EXECUTION AND RESULT ADOPTION

Four lanes classify finite work. Work units execute independently. Only the UI thread may adopt a result.

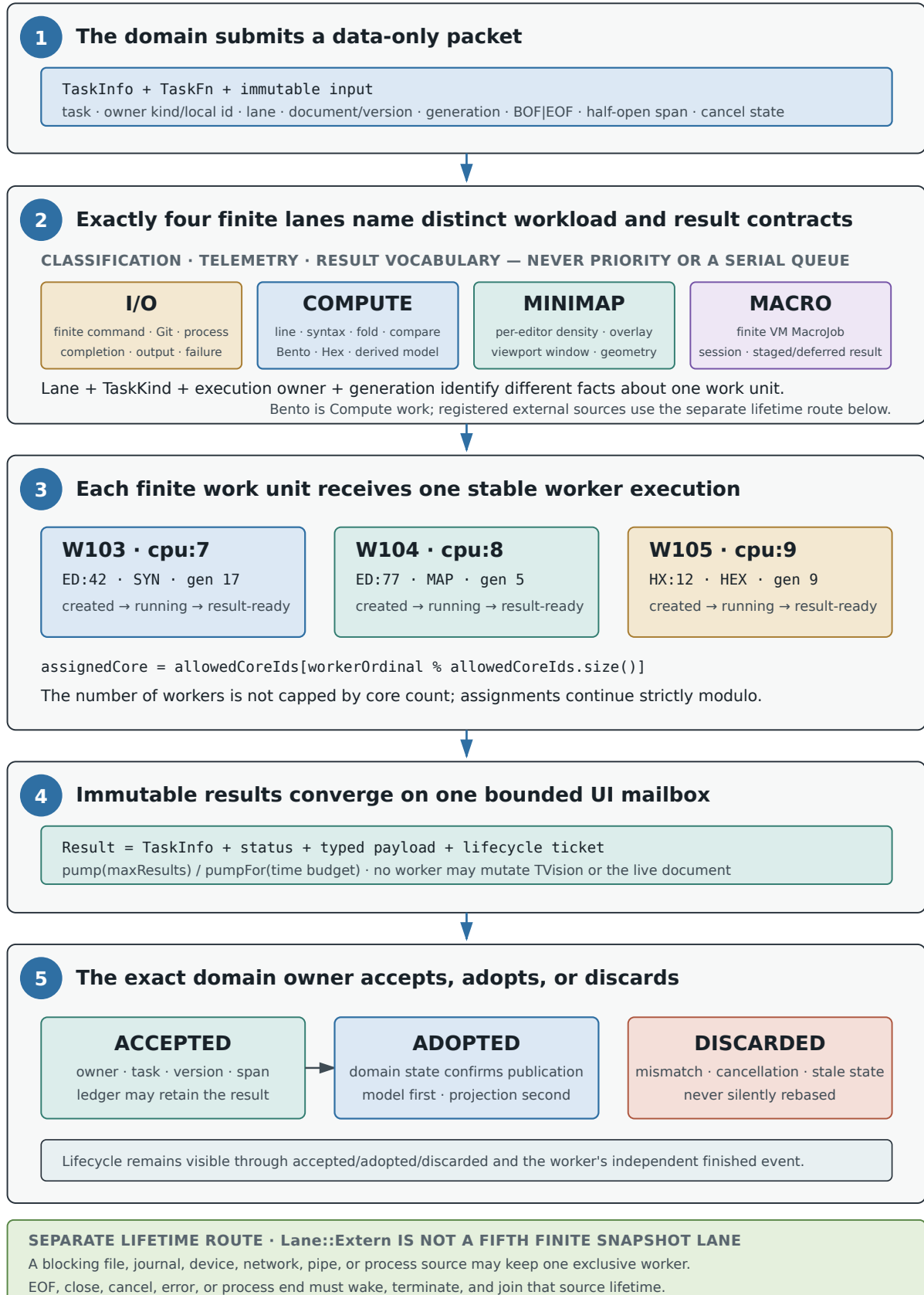


Figure 4.1: Coprocessor execution and result adoption. The four finite lanes classify distinct workload and result contracts; they are not queues or priorities. Every finite work unit receives an independently pinned one-shot worker. Results converge on the bounded UI mailbox, where the exact domain owner accepts, adopts, or discards them. A blocking external source is the explicit lifetime-bound worker exception.

LIFETIME-BOUND EXTERNAL SOURCES

A blocking wait may retain one worker; result routing and UI adoption remain exact and bounded.

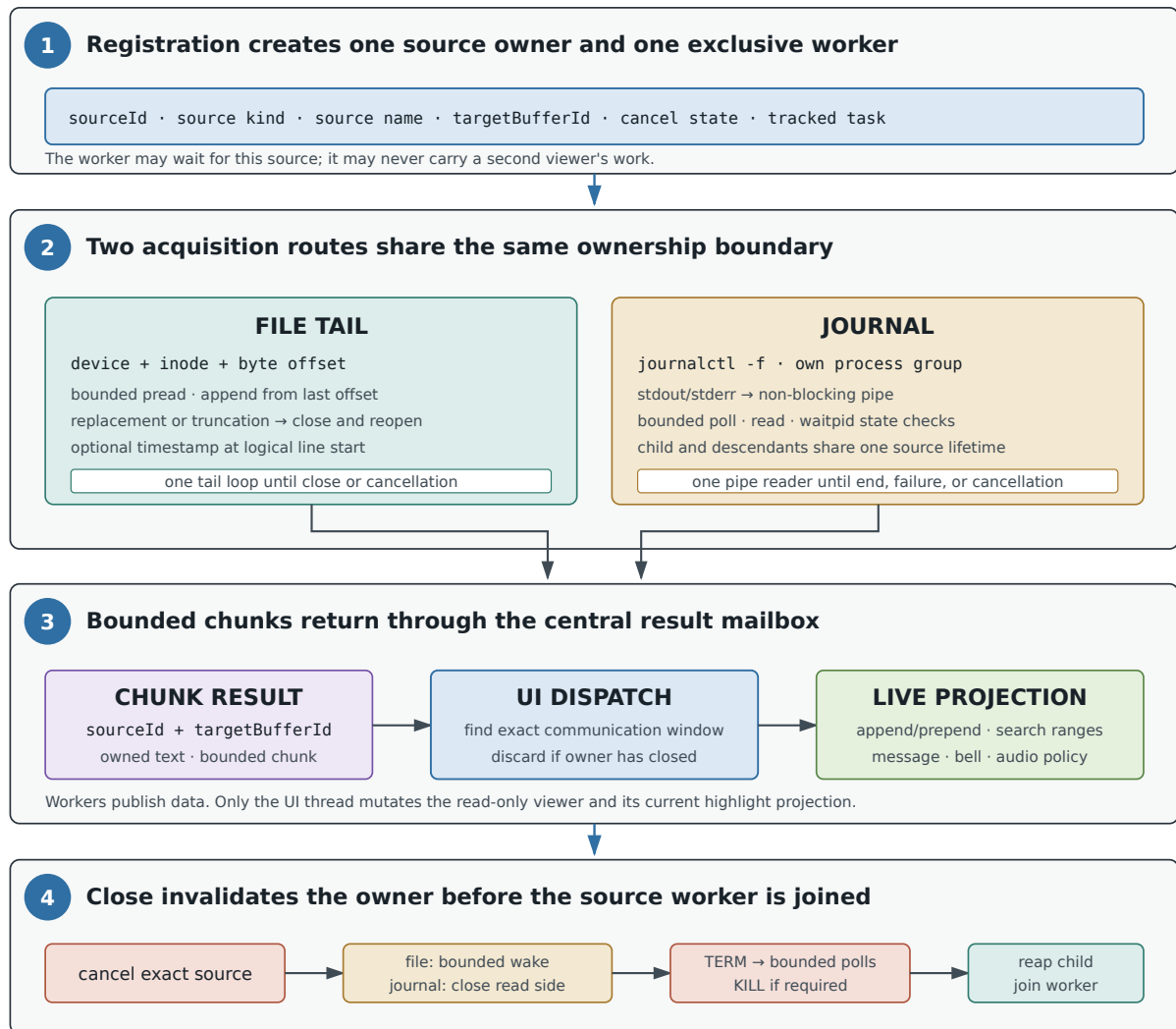


Figure 4.2: Lifetime-bound external-source execution. File tailing handles append, replacement, and truncation through one source-owned reader; journal viewing owns a non-blocking pipe and process group. Both publish bounded chunks through the central mailbox, and close or cancellation invalidates the exact viewer owner before the worker is joined.

Chapter 5

A minimap with eight dots per character

The minimap is a viewport-anchored derived projection in an editor gutter. It has two worker-produced data paths: compact text-density patterns from bounded line windows, and immutable overlay components from selection, search, edit, diagnostic, and File Compare state. The UI thread performs only bounded adoption and ordinary TVision projection. Figure 5.1 on page 14 identifies the three visible components, and Figure 5.2 on page 17 follows the complete data path.

5.1 Geometry and the sampling window

The effective edit settings select **OFF**, **LEADING**, or **TRAILING**, a total minimap width from 2 through 20 cells, and a single-cell viewport-marker glyph. **TextViewportGeometry** reserves the body, marker, and optional separator columns. The body receives all but one cell of the configured width; the marker occupies the independent **infoX** column. The normal and File Compare palettes provide separate colours without changing the generated patterns.

The editor projection request contains body geometry, visible text-row count, viewport width, top line, exact or estimated total line count, document identity and version, and a read snapshot. It is scheduled from semantic and viewport changes, not created by **draw()**. Braille mode is used only when TVision reports the full Braille glyph as one terminal cell. Otherwise the renderer falls back to one block cell per sample.

samplingWindowFor assigns four source-line samples to every minimap row in Braille mode and one in fallback mode. For R minimap rows, its capacity is therefore $4R$ or R lines. A document no larger than that

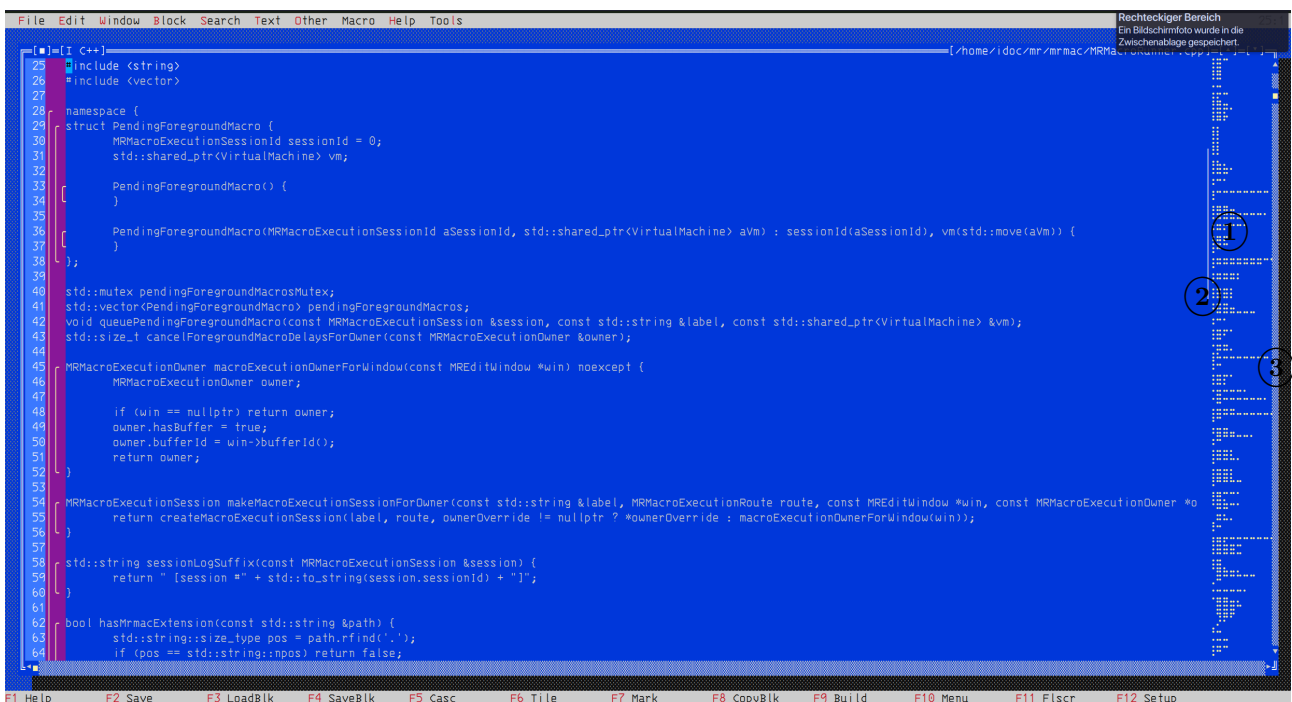


Figure 5.1: Trailing minimap in the editor: (1) the Braille pattern body, (2) the separate viewport-marker column, and (3) the native TVision scrollbar. The minimap is a projection gutter, not a replacement input mechanism for the scrollbar.

capacity uses the interval $[0, totalLines)$. A larger document uses a capacity-sized window centred on `topLine`, clamped first at BOF and then at EOF. The minimap therefore describes the cursor-near region at useful resolution rather than compressing an arbitrarily large document into the same fixed row count.

The projection key contains document identity and version, row and body dimensions, viewport width, renderer mode, and the exact sampling interval. The warm-window ledger retains at most $\max(4, 2 \cdot allowedCoreCount)$ complete, compatible windows. A new directional acquisition covers $\max(8, 4 \cdot allowedCoreCount)$ visible-window capacities and creates at most one finite packet per allowed core. Existing results and reservations are reused span-exactly. Small scroll steps inside this region therefore require neither cancellation nor a new worker wave.

5.2 Worker-side Braille reduction

Each `MiniMapWarmup` packet runs on its own finite one-shot worker in the `MiniMap` lane. The closure owns the read snapshot, geometry, settings, source span and version ticket. Packets read sequentially within their bounded span; no second document model is constructed. The editor-local ledger may accept packets from an older generation of the same version, while only a projection matching the current visible key may replace the canvas.

For every sampled line, the worker walks display characters with TVision's UTF-8 width calculation and the configured tab-stop calculation. Whitespace does not set density bits. Every occupied display-column span is scaled from the editor viewport width into the minimap's horizontal dot columns. Braille mode has twice as many dot columns as body cells; the line mask is a `uint64_t`, so at most 64 horizontal dot columns are retained.

Four line masks and two adjacent dot columns form one Braille cell. Source lines zero through three select the four dot rows; the left column sets bits 01, 02, 04, and 40, while the right column sets 08, 10, 20, and 80. The resulting byte indexes the prebuilt glyph table at Unicode code point `U+2800 + pattern`. The worker does not create glyph strings in its payload. It returns the bytes in `rowPatterns`, plus the document-line start and end represented by every minimap row, the sampling bounds, total lines, geometry, and renderer mode.

5.3 Result adoption and cache continuity

The common dispatcher identifies the exact editor through the task ID before the minimap domain checks owner, generation, document, version, geometry, mode, and span. Matching packets first extend the bounded ledger. As soon as all lines of the current visible subwindow are present, that immutable projection may be adopted even while directional prewarming continues. An older packet may add reusable coverage but may not overwrite a newer viewport projection.

Adoption publishes row patterns and the inverse line mapping and requests a redraw only when visual data changed. Geometry-compatible content may remain visible while a replacement wave runs, but a payload from an older document version is never projected. Multiple editors warm independently, including background windows; their packet waves share neither a focus gate nor a lane-wide worker.

5.4 Bounded reuse and controlled churn

The warm-window ledger is deliberately domain-specific. Unlike syntax, the minimap has no forward lexical state to validate; unlike the line index, it does not establish one global BOF prefix. Its entries are keyed by version, source span, window alignment, viewport and body geometry, renderer mode, and all settings that affect display-column reduction. The bounded window count is the memory policy, not a second document cache.

The first packet implementation proved that worker independence alone is not enough. Continuous `PageDown` and large jumps could create ten to twelve thousand tiny result-ready packets faster than the bounded UI dispatcher could adopt them. Larger directional acquisition windows, span reservations, and early visible-window adoption removed that backlog. In the accepted `enwik9` run, 100 `PageDown` followed by 100 `PageUp` events inside a warmed region created no MAP wave; a distant jump created at most sixteen packets and returned to `MAP:0`.

5.5 Overlay masks and TVision projection

Overlay sources publish immutable, revisioned vectors when selection, search, dirty state, compiler diagnostics, or File Compare state changes. Finite component packets convert these inputs to line masks with the same display width and horizontal scaling as the text projection. A pure selection change therefore recomputes only the search/selection component. Range hashing and document-wide mask construction no longer occur in `draw()`.

For each body cell, `drawGutter` reads the fully adopted text and overlay projections. Colour priority remains fixed: error, warning, File Compare missing, insert, offset, equal, find or selection, changed, then normal. An overlay may colour an otherwise empty pattern cell so that a blank changed or diagnostic line remains visible.

The viewport marker is computed independently from the visible interval within the sampling window and written to `infoX`. It spans at least three rows when the gutter height permits. Finally the renderer writes the Braille glyph, fallback block, separator, and marker into the editor row's ordinary `TDrawBuffer`. Leading and trailing placement use the same projection; the adjacent `TVision` scrollbar remains an independent view with its own event semantics.

MINIMAP WARM-WINDOW AND PROJECTION FLOW

Text density and overlays are immutable derived state; draw() consumes adopted projections only.

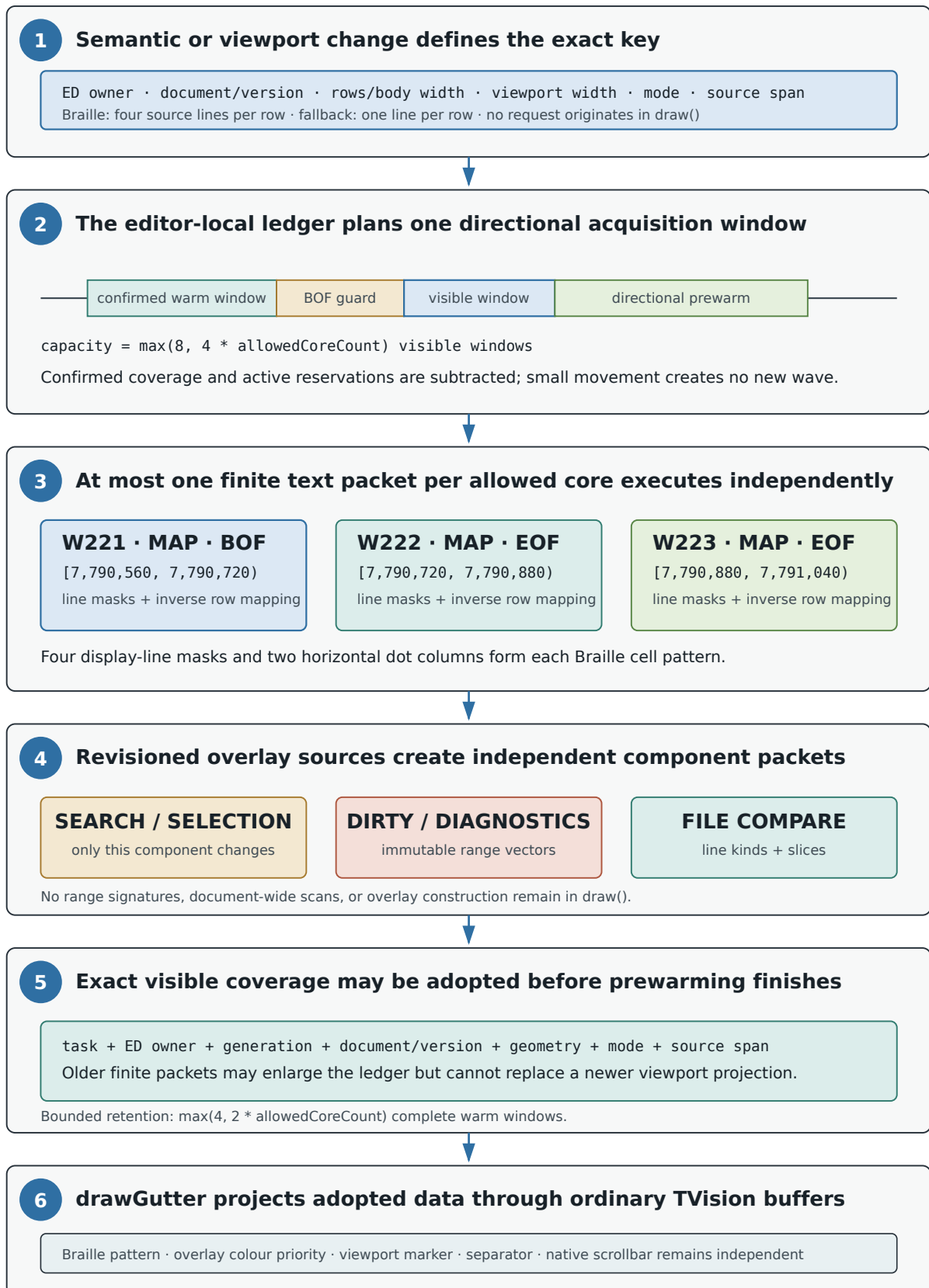


Figure 5.2: Minimap function flow. An editor owner plans uncovered spans from its bounded warm-window ledger. Independently pinned text and overlay packets return immutable projections; an exact visible window is adopted before **drawGutter** writes ordinary TVision buffers.

Chapter 6

A block is geometry, not a clipboard string

The editing tradition behind MR treats a block as a first-class working object. A transient native selection is not sufficient for that role: line, column, and stream blocks have different geometry; a committed block may remain visible after marking has ended; and editing around it must not make its endpoints drift without explanation. The block engine therefore owns a versioned **MRFEBlockGeometry** with mode, lifecycle state, visibility, byte endpoints, line endpoints, and display-column endpoints.

6.1 Two coordinate systems and one explicit lifecycle

The lifecycle has three states. **Inactive** owns no block, **Marking** follows the cursor from a fixed anchor, and **Committed** freezes that geometry while keeping it available for later operations. Hiding a block suppresses its visual projection but does not silently destroy the committed object. Selection and column overlays are derived views of this geometry; neither is an alternative block authority.

Stream and line blocks are grounded primarily in byte ranges. Column blocks are rectangles in terminal display cells and additionally retain their line interval. Converting between these systems uses the editor's configured tab stops and TVision's UTF-8 width rules. A tab or a multi-cell character that only partly intersects a column rectangle is represented by the corresponding number of spaces in its rectangular payload. That rule preserves the visual width of a column operation without pretending that half a character is a valid byte substring.

Each geometry records the document version from which it was normalised. After an ordinary editor mutation, the engine accepts only the matching **DocumentChangeSet**: offsets before the touched range remain stable, offsets after it move by the byte delta, and offsets inside replaced text collapse to the edit start. Column line anchors are then recomputed against the new document. A missing version link is not guessed across; the caller must rebuild or clear the visual block state.

Figure 6.1 on page 19 shows lifecycle, geometry, and mutation as one contract.

6.2 Typed transfer, staged mutation, and the system clipboard

Copy, move, delete, indent, and undent dispatch by block mode. Stream and line operations become range replacements. Column operations first compute one replacement per affected line in display-cell space, then add those replacements to a **StagedTransaction** from bottom to top so earlier byte offsets remain valid. The editor applies that transaction through the same snapshot/version boundary used by other Piece Table mutations. Only after successful adoption does the engine publish the new cursor, selection, overlay, and committed geometry.

Cross-window work uses a **TransferMessage**, not a pointer into the source editor. It carries source and target IDs, copy or move intent, block mode, geometry, row and column dimensions, and an owned byte payload assembled by **MRFEArenaAllocator**. The target interprets that payload according to its own insert/overwrite mode and line-ending context. A move between two documents cannot be one Piece Table transaction: target insertion is accepted first, then the source block is removed. Failure is reported at the boundary; the implementation does not claim cross-document atomicity it cannot provide.

The operating-system clipboard remains the interchange authority. Native editor selections and persistent blocks both project text into TVision's **TClipboard**; paste requests return through the normal TVision event path and respect read-only targets. MRMAC exposes the same system clipboard operation rather than maintaining a second macro buffer. The richer block geometry remains editor state, while the clipboard contains only transferable text. That separation is why compatibility terminology such as “buffer” does not become a competing storage model.

BLOCK GEOMETRY AND STAGED MUTATION

The persistent block is authoritative; native selection, overlays and clipboard text are projections.

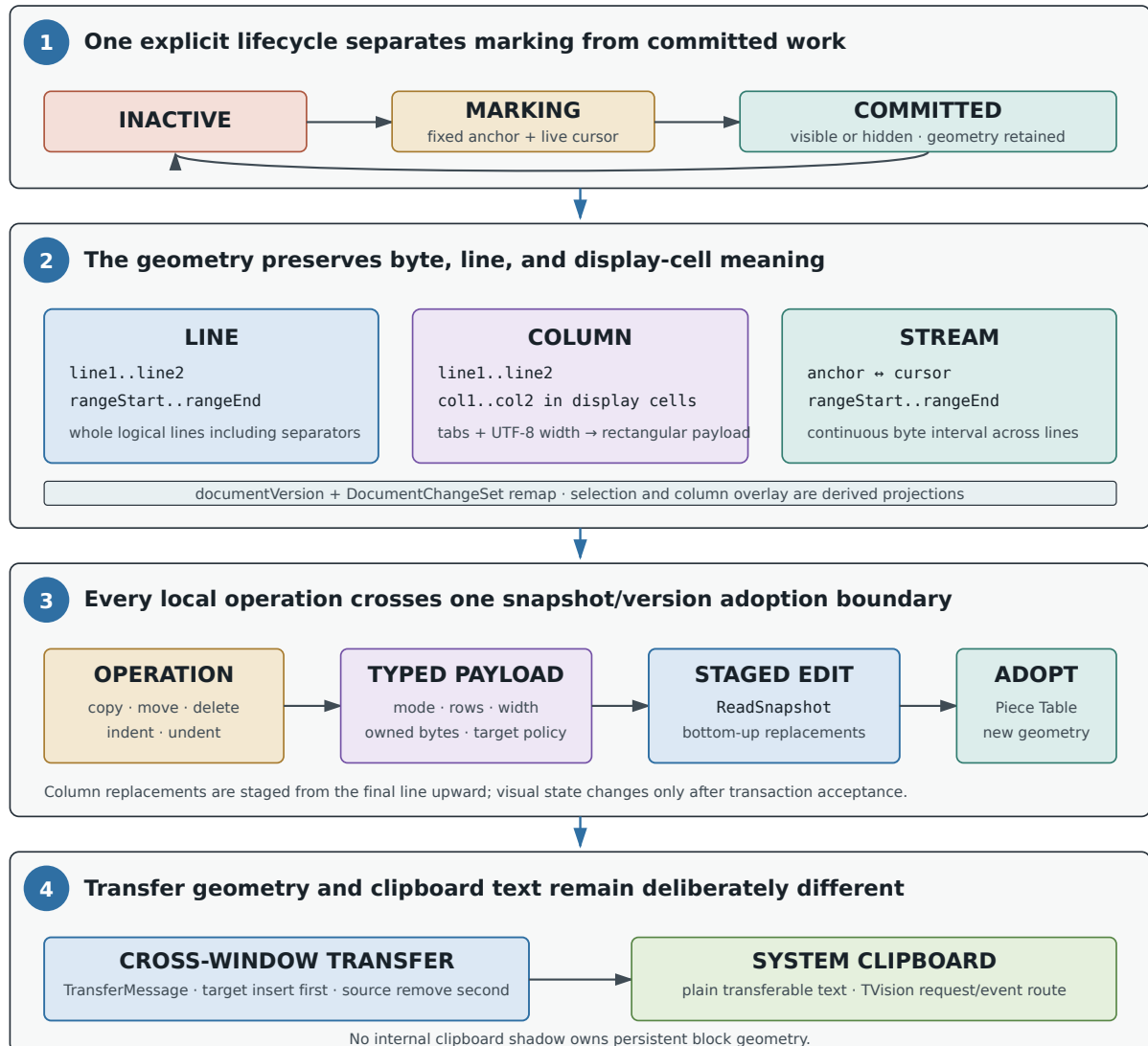


Figure 6.1: Block geometry and mutation. Marking produces one versioned line, column, or stream object; selection and overlays are projections. Operations capture a typed payload and build a **StagedTransaction**, which the editor adopts against its snapshot before publishing the new committed geometry.

Part III

A coordinated workspace

Chapter 7

Bento: one task, several views

Bento is perhaps the strongest UI idea in MR. It did not begin as an attempt to invent a generic pane framework or a tiled window manager for the terminal. The recorded starting point was the split editor. A split made an immediate editing need visible: one document, two places in that document, and both places still part of the same editing task. Stabilising those viewports then exposed the more general question.

A compiler failure calls for source, build output, and a problem list that lead to one another. File comparison requires two documents, their difference structure, and change navigation. A macro debugger needs source, output, variables, and watches around one Execution Session. These are not simply collections of windows. They are coordinated perspectives on one working object. Loose top-level windows would make their focus, geometry, lifetime, and restoration the user's problem. The Bento step was to give the task one desktop object and let that object own the spatial relationship between its perspectives.

MRBentoBox is that object. Its layout is a tree: internal nodes split an area horizontally or vertically, while leaves carry a concrete pane role. The active leaf, a temporarily maximised leaf, visibility, and divider positions are part of the same layout state. The box owns this tree, pane lifetime, standard Bento chrome, and the routing of focus and events. The leaves are nevertheless ordinary TVision pane windows rather than drawn imitations of windows. This preserves TVision's ownership of focus, Z order, drawing, and lifetime while giving the user one coherent workspace.

The difficult part is not dividing a rectangle. A Bento is one TVision desktop window with several possible child pane hosts, and it must make that plurality feel like one editor rather than a fight for focus. The box keeps one explicit active leaf. A mouse click first identifies its leaf in Bento-local coordinates; changing leaves completes transient input in the previous pane, selects the new pane host and its editor, and projects the focused frame. **Ctrl-Tab** follows the same active-leaf path. Focus is therefore neither guessed from drawing order nor delegated to a private second event loop.

Event routing is just as deliberate. The outer box handles its own close frame, title drop-lists, splitters, and workspace commands before it passes an event on. A divider hit starts a constrained divider drag; a content click or wheel event is directed to the leaf under the pointer, while a keyboard event uses the active leaf. Pane chrome is drawn by dedicated frame views, and each pane keeps its own local editor, scrollbars, and policy. In particular, the container does not simulate child windows by writing directly to the screen. It uses normal TVision views, but gives them an explicit routing discipline.

7.1 Dirty projection without a second UI model

Rendering needs a second discipline. Content, pane chrome, scrollbars, layout, and overlays have independent dirty reasons. The Bento projection pump coalesces those reasons; a layout change dominates narrower redraws, whereas a cursor or selection change need not reconstruct the whole tree. Semantic changes occur in their owning document, diagnostics, debugger, or compare model first, and only then mark the relevant projection dirty. This is why a compound TUI view remains responsive without turning repaint state into a second hidden model.

The dirty mask is consumed at controlled event and adoption exits. A flush first takes and clears the accumulated mask. If layout is dirty, `layoutSplitPanels` performs the complete native TVision projection, updates an associated Sidekick overlay if required, redraws open drop-lists, and returns; content, chrome, and scrollbar bits need not repeat work already dominated by layout. Without layout, the pump independently projects shared editor content, source scrollbars, Sidekick state, and pane frames. Window close suppresses the pump because view lifetime has already become the dominant operation.

Figure 7.1 on page 22 makes the crucial negative space visible: dirty bits schedule projection work, but they do not store focus, document, diagnostics, compare, or debugger state.

BENTO DIRTY-PROJECTION PUMP

Semantic state changes first. Dirty bits only coalesce the native TVision work needed to show it.

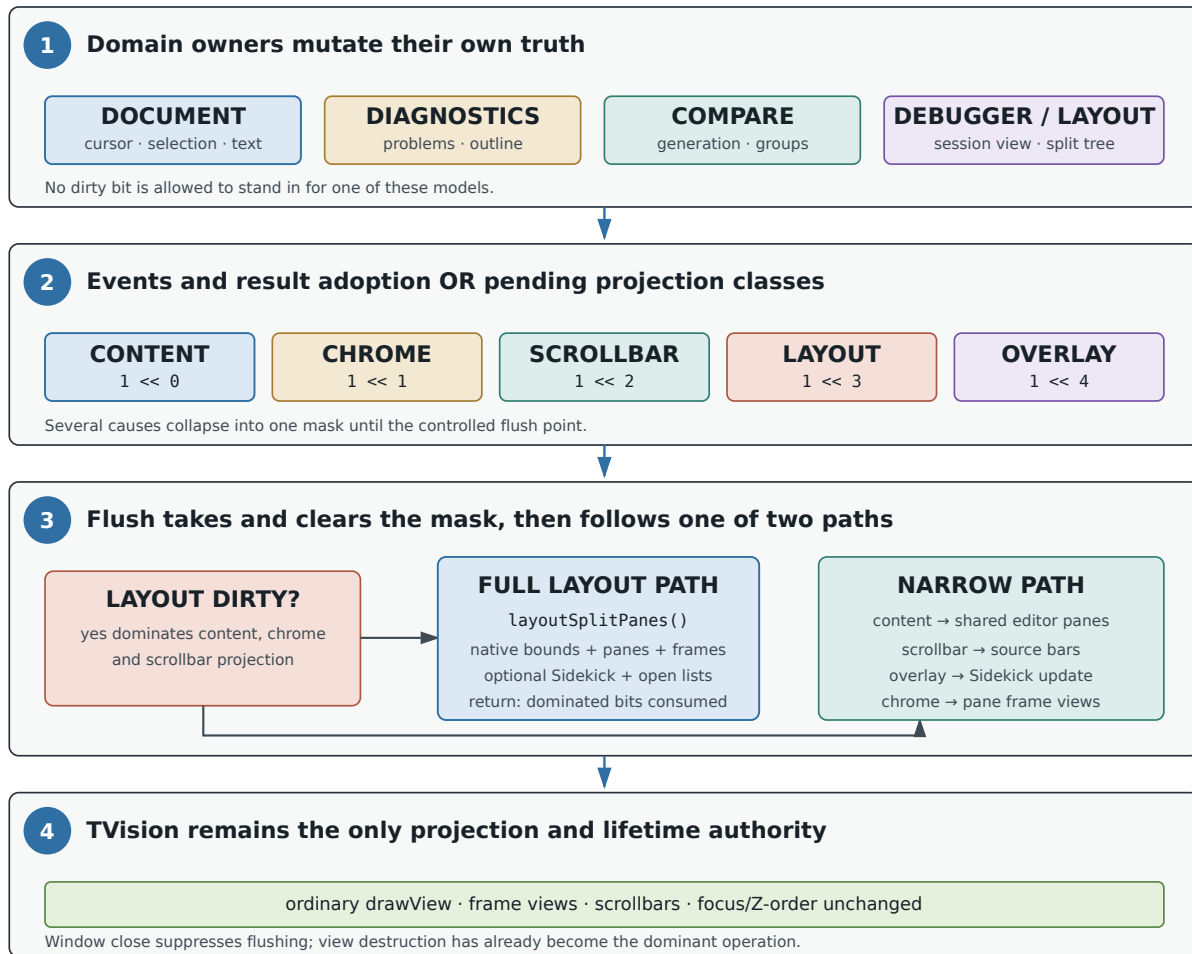


Figure 7.1: Bento dirty-projection pump. Semantic owners change first and mark only pending projection classes. A controlled flush coalesces the mask; layout dominates narrower redraws, while non-layout work reaches only the required native TVision views. The mask is scheduling state, never domain truth.

That boundary made one construction serve several rather different tasks. In document-viewport mode, editor leaves share the source buffer so that distant regions of one document can be edited together. The compiler workspace brings source, output, and diagnostics into a navigable unit. File Compare joins two sources to their diff state and change operations. The debugger arranges its source, output, variables, and watches around the session it observes; the workspace does not become an alternative owner of debugger truth. Bento snapshots preserve the layout tree, leaf roles, dividers, active and maximised leaf, so a workspace can resume the working arrangement rather than merely reopen a pile of files.

Bento 1.5 adds a closed widget mask to every persisted leaf. The first two bits mean Fold Gutter and MiniMap; unknown bits are rejected rather than silently reduced. The existing `WORKSPACE` payload serialises leaf ID, role, visibility, and mask, while the older `bento=v1` form remains readable with role-specific defaults. Fold spans, minimap projections, ledgers, owners, tasks, workers, and cores are never persisted. Restore reconstructs the task arrangement and creates fresh runtime owners.

Problems, Structure, and Functions now have pane-owned finite projection tasks. `BDIAG` parses and remaps compiler diagnostics from the snapshot captured at build start; exactly one terminal parse follows the completed external command. `BOUT` performs outline sorting, hierarchy building, offset resolution, and text formatting from a consistent fold snapshot. Each pane has at most one active worker and one coalesced newest request. Adoption checks the exact pane owner, generation, role, documents, versions, and projection hash before changing text, markers, or status. Reentrant title and minimap broadcasts are held behind the complete adoption boundary.

Structure and Functions request a complete fold basis only for documents of at most 20,000 lines and eight MiB. Above either bound they deliberately project the confirmed visible fold spans and report a partial outline. This is a named resource boundary, not permission to infer unvalidated structure. Diagnostic remapping likewise

requires a complete observed change chain; an untracked source commit invalidates old diagnostics until the next build establishes a new common snapshot.

The Hex editor supplies the sharpest proof of the design. It needs six simultaneous projections of the same bytes—Hex, Strings, Inspector, Decimal, Binary, and Octal—and one canonical byte cursor. The first integration was discarded because it taught `MRBentoBox` about Hex-specific branches, gave the primary leaf exceptional treatment, and tried to repair chrome after drawing. The successor, `MRBentoHexEditor`, is a specialised Bento subclass: it reuses the generic layout, pane, focus, and chrome mechanisms, while owning byte roles, byte cursor, and number-format semantics itself. That division is what lets Bento be a powerful UI element rather than a container that gradually learns every feature it contains.

Each of those six panes is also its own `HexPane` execution owner. Finite workers prepare immutable offset rows, cell text, string-search halos, or the Inspector’s value rows from a `ReadSnapshot`. The draw path consumes that payload and a bounded cursor/edit overlay; it performs no string search, numeric conversion, or complete pane formatting. At most one request is active per pane, and a full viewport change therefore has a six-worker ceiling. Cursor movement inside an adopted viewport reuses all five data-pane projections; only the cursor-dependent Inspector may require new derived work. All editable data panes share one byte cursor and one record/column anchor, while TVision supplies the native cursor only to the focused pane. The logical record index uses `size_t`, but the native TVision scrollbar exposes an `int`; fully proportional scrolling above `INT_MAX` records is therefore an explicit support boundary.

7.2 File Compare as an ordered finite pipeline

File Compare uses a separate ordered pipeline rather than hiding large copies inside UI adoption. Two pane-owned workers acquire and split the source snapshots, one worker computes and normalises the Myers edit path, and two pane-owned workers build the immutable original and compare projections. Thus one generation is exactly a $2 \rightarrow 1 \rightarrow 2$ sequence. The Myers frontier orders only that comparison; it does not serialize other Bento owners or the Compute lane.

The two acquisition results must name the active generation and the exact document/version pair before the ordered diff is submitted. Its result carries both source identities and normalised change groups. Each projection worker then derives the pane text, line-kind map, and minimap slices for one side. The UI publishes the comparison only after both immutable pane projections have arrived with matching generation, source version, and editable-mode policy. A stale result cancels the generation instead of rebasing its hunks onto changed text.

Navigation walks the adopted change groups without creating workers. Applying a change uses the current group to build one ordinary editor mutation, marks the old comparison stale, cancels any remaining tasks, refreshes the affected snapshot, and starts a fresh five-task generation. Responsiveness therefore does not come from an approximate diff; it comes from moving acquisition, Myers computation, and display projection outside the UI thread while keeping adoption exact.

Bento is not the whole editor family, and that distinction is deliberate. `MRSidekickEditor` does not inherit from Bento; it is a `TView` for short-lived, code-relative editing. In snippet mode it edits a proposed replacement range and supports placeholder navigation. In read-only mode it places explanatory, reference, or diagnostic text beside or beneath the code that prompted it. A Sidekick belongs to its parent buffer and disappears with that context; it neither owns a durable workspace nor needs a layout tree. The two forms demonstrate an important design judgement: a persistent task with several simultaneous perspectives needs Bento, while a brief local edit or explanation needs an anchored editor of its own.

FILE COMPARE: ONE FINITE 2 → 1 → 2 GENERATION

Only the Myers frontier is ordered. Other Bento owners and Compute work remain independent.

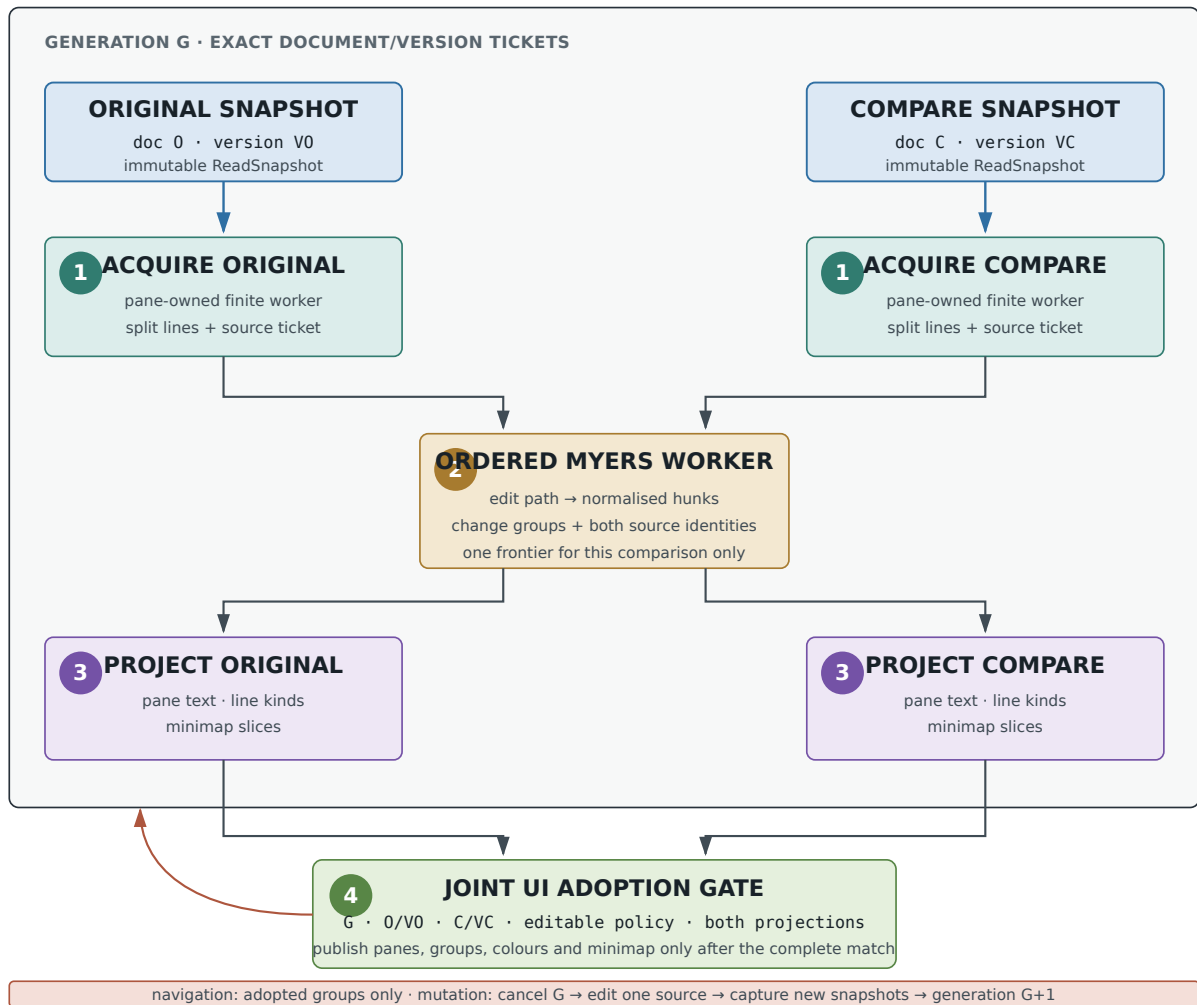


Figure 7.2: File Compare's finite 2 → 1 → 2 generation. Two pane-owned acquisition workers meet at one ordered Myers worker; two immutable pane projections return to a joint adoption gate. Navigation is worker-free, while either-side mutation invalidates the generation and restarts it from new snapshots.

Part IV

MRMAC as an integrated runtime

Chapter 8

A language that belongs to the editor

MRMAC is not merely a file of macro commands interpreted at startup. Its canonical compiler produces in-memory bytecode, while the Macro Catalog owns loaded files, metadata, profiles, and resident bytecode. That decision joins extensibility to a clearly identifiable runtime owner.

8.1 One compiler, three deliberate products

The earlier Flex/Bison path and the hand-written compiler once formed two possible sources of language truth. Consolidation removed that dual track. `mrmac.c` now contains the canonical scanner, parser, type checks, symbol handling, and bytecode emission used by every execution route. Opcode semantics remain a VM contract; neither the debugger nor an editor feature injects an alternative bytecode dialect.

On-demand compilation is its normal economy. A binding may name a file that has not yet been loaded; on first use, the source is resolved, canonically compiled, and then retained in memory. This avoids a large startup burden without parsing the source again for every execution. An index for warming bound macro files in advance also exists. The infrastructure is present, but the current application call path is not connected to it. This manual therefore does not claim active “pre-use compilation” as a delivered feature.

The same restraint applies to debug information. Source maps are created on the debugging path, not for every ordinary macro execution. There is no second, secret debug compiler. One compiler, one bytecode model, and additional cost only where the user actually requests it.

The third product is a watch expression. It reuses the canonical expression parser and bytecode emitter with an explicit symbol/type environment and `watch_expression` restriction. Procedures, assignment, UI work, and file or process side effects are rejected; only the bounded intrinsic allow-list is available. The result still ends in ordinary VM bytecode and `OP_HALT`. Pure-expression mode is therefore a constrained entry into one compiler, not a small language implemented beside it.

8.2 Hash handles, chained lvalues, and scope ownership

An MRMAC `Value` does not embed a nested hash tree. It carries a type, an integer hash handle, and the storage domain of that handle. Every VM has a local `MRVMHashStore`; macro-visible application state uses the global store behind the central K/V roots. A chained read repeatedly resolves the current handle and key. A chained assignment keeps each intermediate hash as a concrete value and applies the final store to the resolved leaf, so `root['C']['KEY']` is a real lvalue rather than a reconstructed delimited name.

Crossing a storage boundary is explicit. Copying a local hash into global state recursively clones nested hashes and hashes inside typed arrays into the target store with an identity map that preserves cycles, and retags their storage domain. A global handle is never left pointing into a VM-local store that will disappear at macro completion. Unreachable local trees can consequently be cleared independently, while approved central roots and explicit application globals retain their intended runtime lifetime.

Scope is orthogonal to source files. Finite `DEF_*` values belong to the VM; closure values belong to the recurring closure identity; long-lived session values belong below `EXECSSESSIONS`; explicit application globals live below `MACROGLOBALS`. There is no file-global namespace. The debugger projects these actual stores and mutates them only through the VM’s validated value path. Hashes are volatile runtime structures: neither settings nor workspace persistence serialises them.

Figure 8.1 on page 27 joins the compiler and value-store boundaries that make these language features usable by normal macros, Execution Sessions, MMP, and the debugger.

MRMAC COMPILER AND VALUE OWNERSHIP

One compiler produces ordinary bytecode, optional debug mapping, and restricted watch bytecode.

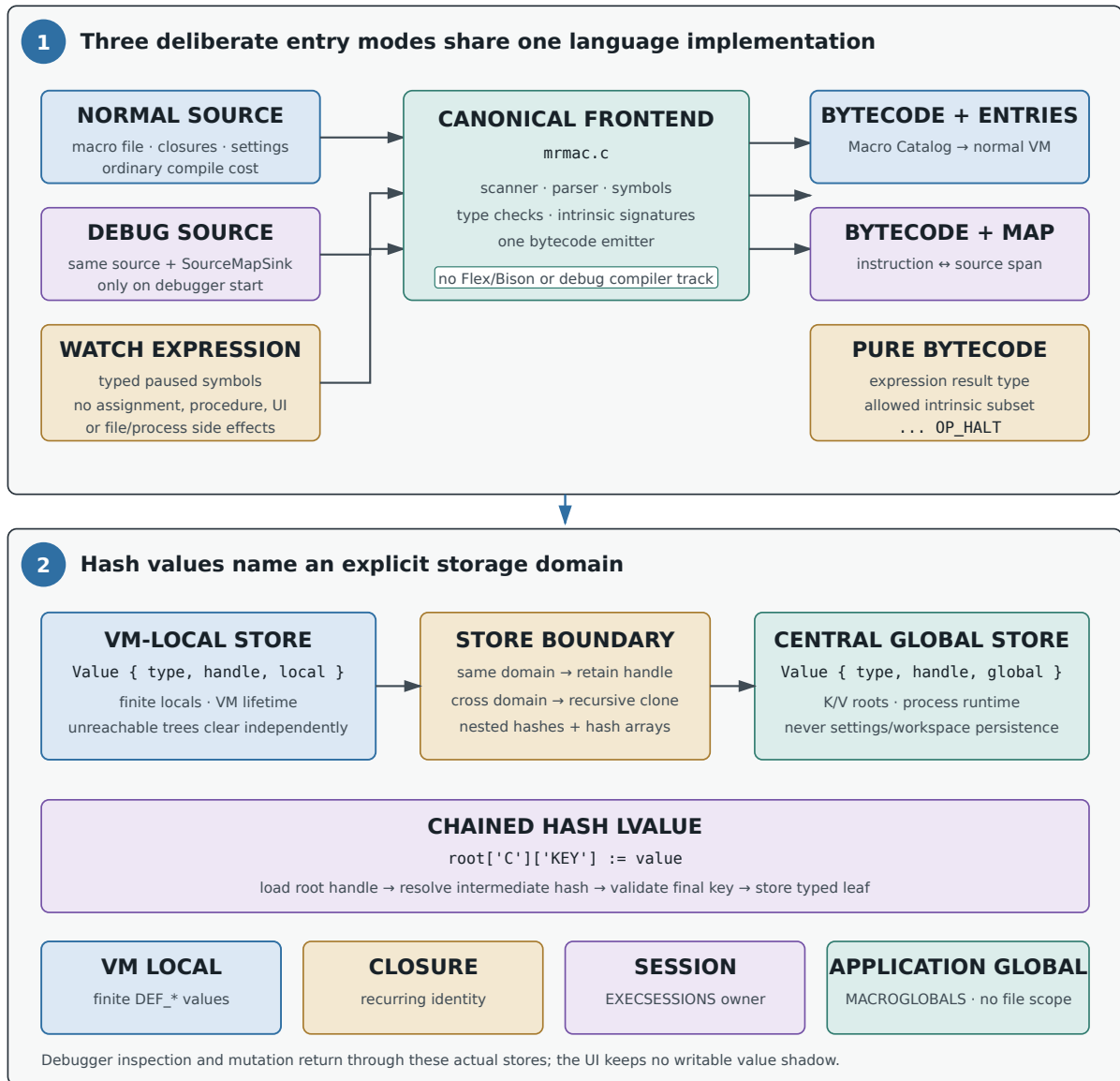


Figure 8.1: Canonical compiler products and runtime value ownership. Normal, source-map, and pure watch compilation share one parser, type system, and bytecode emitter. At runtime, typed values resolve local or central hash stores explicitly; crossing that boundary clones nested containers instead of exporting a per-VM handle.

Chapter 9

Inside the MRMAC virtual machine

The virtual machine is the point at which MRMAC stops being source text and becomes editor behaviour. It is a compact stack machine, but “stack machine” is only the beginning of the useful explanation. The VM also gives values their storage lifetime, separates local and process-wide runtime state, dispatches typed language operations into the application, preserves a paused execution, and produces results that another owner may adopt. Treating it as a loop over opcodes would miss most of its contract.

Figure 9.1 on page 33 follows one compiled macro through that contract. The compiler, catalog, runner, VM, runtime K/V tree, and UI thread remain distinct owners. In particular, the diagram does not turn the VM into a general application object registry: macro-visible runtime values live in the VM storage model, while live TVision objects and document adoption stay outside it.

9.1 The bytecode image and execution loop

`mrmac.c` emits one byte stream containing executable units and their entry offsets. Integer and real literals are encoded directly behind their push opcodes; strings and symbolic operation names are zero-terminated operands. Branch and call instructions carry bytecode offsets. The loaded image is therefore not an abstract syntax tree retained for later interpretation. It is the canonical executable form held by the Macro Catalog.

VirtualMachine::BytecodeExecution starts at a selected entry offset and maintains an instruction pointer, an operand stack, and a return-offset stack. Push, load, arithmetic, comparison, logic, hash, and array instructions transform typed operands. **GOTO** and **JZ** alter the instruction pointer; **CALL** saves the next offset before entering its target, and **RET** restores it. Invalid targets, stack underflow, malformed operands, and type mismatches become VM errors rather than unchecked jumps through the byte stream.

This design makes the hot path direct. There is no node allocation or visitor dispatch per source construct, and a normal run does not carry source spans beside every instruction. The debugger requests a source map from the same compiler only when a debug start needs the relation between instruction offsets and source spans. Watch expressions use that frontend in a restricted expression mode and end in **HALT**; they are not handed to a second parser or a general-purpose evaluator.

The complete bytecode run holds the recursive VM execution lock. This is a semantic boundary, not a claim that a Macro-lane worker is the only worker in the coprocessor. Runtime environment fields and the global K/V tree are process-wide, so two independently owned **MacroJob** workers must not mutate them concurrently. Worker identity, core assignment, and result delivery remain independent even where VM state access itself is serialised.

9.2 One value model, several lifetimes

A VM value records its type and the matching payload: integer, real, string, character, array, or hash handle. Arrays retain typed elements. A hash value also records whether its handle belongs to the VM-local hash store or to the central global store. That storage bit is small but decisive: a numeric handle without its storage domain would not identify a value.

Finite **DEF_*** variables normally live in the VM’s variable map and local hash store. Clearing the execution clears unreachable local value trees. A recurring closure instead restores its declared values from its closure state, and a long-lived Execution Session restores session variables from **EXECSESSIONS**. These are language lifetimes, not namespaces inferred from source file names. Two macro files do not acquire private globals merely because they came from different paths.

Process-wide macro state uses the central runtime K/V store. Its named roots separate responsibilities: **MACROGLOBALS** contains explicit application globals, **MACROCATALOG** contains loaded macro and bytecode metadata, **EXECSESSIONS** contains session and closure state, **MODELESSUI** contains retained modeless UI values, and **MACRODEBUGGER** contains debugger definitions and live debugger bookkeeping. **APPLICATIONUI** contains application-owned transient UI coordination such as the message-line Static Mode semaphore. These roots share a storage substrate, but they are not one undifferentiated dictionary.

Static Mode illustrates the separation between semantic coordination and view mechanics. Its K/V semaphore becomes authoritative before the menu and status lines transition. Entry clears ordinary message owners, suppresses new transient messages, and permits only the numeric progress projection. The function-key labels animate towards **Esc Abort**, but F1 through F12 cease dispatch immediately rather than waiting for the animation. Exit clears the progress projection and restores the captured labels through the inverse transition. The views retain animation mechanics; they do not own the mode. Fullscreen temporarily suspends both the projection and the event restriction without clearing the semaphore. F11 therefore remains an unconditional presentation command, Escape first leaves Fullscreen, and a transient F10 menu keeps the Static Mode lane blank. If the operation still runs after Fullscreen, the retained mode and progress become visible again.

Hash and hash-array values may cross a storage boundary. In that case the VM copies the reachable value graph into the target store with a source-to-target handle map, preserving cycles and shared subgraphs, and marks the resulting handles with the target domain. Within one store, it may retain the existing handle. This prevents a global value from referring accidentally to a local hash that disappears when a finite VM ends. It also explains why C++ records used during transfer are not allowed to become a second value-bearing registry beside the K/V tree.

9.3 Opcodes end at typed application boundaries

Pure bytecode operations are only one instruction family. **OP_INTRINSIC** carries a symbolic name and argument count, pops the arguments in source order, and pushes the typed result returned by **MRVMIntrinsics**. Hash inspection, string conversion, environment queries, search operations, and typed UI inquiries enter through this value returning boundary.

OP_PROC similarly carries a name and arguments, but dispatches a side-effecting procedure. The procedure catalog classifies the operation into configuration, runtime, editor, or macro families before the corresponding executor handles it. Variable procedures such as tab expansion have a separate opcode because their destination is an existing VM variable rather than an ordinary return value. System variables are reached through the normal load and store instructions, which gives the VM one place to enforce read-only state, type rules, and route restrictions.

Symbolic names in bytecode are deliberate. They keep the compiler's signature tables, the execution profiler, and the VM dispatch vocabulary aligned without exposing C++ function addresses as language state. They do not make the calls untyped: the compiler checks signatures, and the VM validates the values again where runtime state determines what is legal.

The VM is consequently integrated, not sandboxed. An editor procedure may move a cursor or stage a text mutation; a runtime procedure may start another macro; a typed UI procedure may update retained modeless state. Integration does not erase ownership. A worker-side VM receives immutable or copied input and records a result; it does not gain permission to mutate a TVision view or publish a document version.

9.4 The execution profile chooses the legal route

Before execution, **mrvmAnalyzeBytecode** scans the same byte stream and classifies its opcodes, intrinsics, procedures, and system-variable accesses. Pure computation is background-safe. Editor and TVision-affine operations require the UI thread. A bounded subset of editor operations is eligible for staged background execution, while external I/O remains explicitly visible in the profile. This analysis is conservative: an unknown or unsupported operation does not become background-safe by optimism.

The runner turns that profile into the routes whose session lifetimes are shown in Chapter 10. At the VM boundary, a UI-affine macro may use live editor state, a background-safe macro becomes a finite **MacroJob**, and a staged-background macro receives a document snapshot, cursor and selection state, relevant runtime values, and a conflict basis. Its VM writes a staged edit transaction and deferred UI commands as data.

The route therefore changes the available projection of the application, not the bytecode language. The same opcode may read a live system variable on the UI route or its captured staged equivalent on the worker route. Unsupported UI-affine operations are rejected from staged execution instead of quietly calling across the thread boundary. The UI thread accepts a staged result only while its owner and conflict basis still match, changes

the model before projecting deferred UI effects, and discards the whole result on mismatch. There is no partial repair or implicit rebase.

9.5 Yielding, debugging, cancellation, and completion

A foreground `DELAY` and a debugger stop both suspend execution without pretending that the bytecode has completed. The VM preserves the bytecode image, instruction pointer, call stack, return values, parameter string, and the route-specific state needed for continuation. A debug stop additionally records its source identity, stop reason, breakpoints, step policy, and live variables. Continuing resumes that same execution state; it does not replay the macro from its first instruction.

The parked VM belongs to the logical Execution Session. It does not justify a blocked coprocessor worker while a user inspects variables or while a delay waits for its deadline. A later scheduler or debugger action creates the next finite execution action. This distinction is what lets session lifetime be longer than worker lifetime without turning the Macro lane into a collection of sleeping threads.

Cancellation is cooperative. The bytecode loop checks the active cancellation request at instruction boundaries, and longer operations must provide their own bounded checks where interruption is supported. The VM does not terminate a thread at an arbitrary machine instruction. Likewise, a debugger pause is a request observed at a safe VM boundary; an intrinsic or procedure is not half-applied merely to make the pause appear instantaneous.

Completion yields typed effects rather than ownership transfer. A foreground run leaves its return state and log with the caller. A background run returns log lines and explicitly requested UI commands. A staged run returns its transaction, state deltas, deferred UI commands, and conflict basis. A debug run returns a stop or completion snapshot while the live paused VM remains owned by its session. In every case, the consumer decides whether and how that result becomes application-visible.

9.6 Bytecode calls and macro calls are different operations

`CALL` and `RET` are intra-image control-flow instructions. They use the current VM, variable map, operand stack, and bytecode image, and their return stack contains byte offsets only. They are suitable for compiler generated control flow whose target is already part of the executable image. No catalog lookup, source resolution, or new macro identity occurs at that boundary.

`RUN_MACRO` is a language-level macro invocation. Its file-qualified specification is normalised and resolved through the Macro Catalog, which provides the target file, entry offset, attributes, execution profile, and resident bytecode image. The invocation creates a child VM, supplies the parameter string, propagates the current Execution Session identity, and selects closure state when the target is a closure unit. The child's log is joined to the caller's log after execution; transient and dump attributes are then applied to the catalogued image rather than to the caller's bytecode stack.

This distinction prevents a source-level macro call from becoming an unchecked offset jump into whichever image happens to be resident. It also keeps `FIRST_RUN` meaningful per catalogued macro definition and keeps parameters scoped to the invoked macro. The parent parameter context is restored after the child finishes, while return and error outputs propagate through the guarded execution state.

Staged background execution tightens the boundary. It cannot perform an unplanned lazy file load from a worker. A child macro must already be resident and included in the captured staged execution context; otherwise the call fails through the normal runtime error path. This avoids turning a seemingly finite staged transaction into hidden filesystem I/O and prevents its execution basis from changing after submission.

Debug execution preserves the same distinction. Stepping into `RUN_MACRO` starts a child debug VM with the target's own source map. If the child stops, the debugger retains a child frame together with the parent instruction offset and paused parent VM. Continue and step-out can therefore return to the real parent execution rather than reconstructing a synthetic source stack from macro names.

9.7 Authority, persistence, and failure containment

Several parts of this path deliberately know less than their neighbours. The compiler knows symbols, signatures, source positions, and bytecode layout, but does not own runtime values. The Macro Catalog knows loaded files, definition metadata, entry offsets, profiles, and resident images, but does not own a live instruction pointer. A VM owns one executable state, while an Execution Session owns the logical identity and lifetime in which that state runs or is parked. The UI consumes results and debugger projections, but does not keep a writable copy of VM truth.

The runtime K/V store extends beyond one VM without becoming persistence by default. Explicit macro globals, modelless values, session state, catalog metadata, and debugger state are process-runtime data. The workspace boundary in Chapter 19 consequently persists domain state, never a VM instruction pointer, stack, live hash handle, source map, or paused execution.

Settings are a deliberately narrower exception to the usual runtime-only story. Chapter 14 describes the controlled MRSETUP route from persisted macro source to the authoritative settings model. It does not make general VM K/V roots serialisable or turn a runtime global hash into an alternative settings file.

TVision remains on the other side of the same boundary. Typed MMP procedures may write retained macro-visible values under MODELESSUI; the projection described in Chapter 12 leaves live windows, focus, Z order, event dispatch, and drawing lifetime with TVision. A parked VM never keeps a view pointer alive.

Runtime failure is contained at the execution boundary. Invalid bytecode operands, type errors, bounds errors, unknown operations, and exceptions from application procedures end the current run and produce a VM error line. A background result consequently carries failure instead of throwing through the result mailbox into the UI loop. The manual does not promise stable numeric error codes where the implementation provides diagnostic text and route state.

There are also explicit costs. The process-wide execution lock limits concurrent VM mutation even when the coprocessor can run many unrelated non-macro tasks in parallel. Cooperative cancellation cannot interrupt the middle of an opaque operating-system call. Staged execution supports only the operations for which captured input and an accountable commit can preserve semantics. These limits are preferable to three misleading claims: that a worker implies concurrent VM state, that every procedure can be rolled back, or that every running macro is safe to move off the UI thread.

9.8 Why this is a VM rather than a callback catalogue

A catalogue of application callbacks could expose editor operations with much less machinery, but it would not provide a language runtime. Control flow, expressions, typed locals, arrays, nested hashes, macro calls, closures, sessions, and debugger continuation would then have to be recreated by the host around every callback. More importantly, no single executable object would exist that could be profiled before execution or parked at an exact instruction.

The bytecode boundary makes source-independent questions answerable. The runner can count opcodes and named calls, identify UI affinity and external I/O, and decide whether staging is possible without executing the macro. The catalog can keep an immutable resident image and exact entry offsets. The debugger can bind a source span to a bytecode offset and later verify that a paused VM is still stopped at that offset. These capabilities follow from one canonical representation; they are not metadata heuristically reconstructed from a series of callbacks.

The VM also makes extension disciplined. Adding an intrinsic or procedure is not merely adding a C++ function to a public registry. Its compiler signature, runtime type checks, execution-profile classification, background/staging behaviour, error semantics, and documentation must agree. A typed operation may be powerful—it can start a process, mutate a document, or describe a modelless window—but the instruction still crosses a named point at which those obligations can be reviewed.

This is why direct TVision escape hatches would be particularly damaging. Passing a view pointer or paint callback through a generic VM value would evade storage lifetime, route profiling, staged-result validation, debugger inspection, and workspace rules at once. Logical identifiers and retained typed values are less superficially flexible, but they keep a macro explainable after the original call has returned.

Nor is bytecode treated as a portable interchange format. Opcode numbers and operand layout are an internal compiler/VM contract in the current executable and repository. The Macro Catalog may retain a compiled image for the process lifetime, but settings and workspaces persist source-level or domain-level state, not an opaque VM snapshot. This keeps evolution tied to the canonical compiler rather than creating a bytecode migration problem beside it.

9.9 Observability without a shadow runtime

The VM exposes several different forms of evidence because they answer different questions. Its execution log records important notices and failures; ordinary instruction tracing can be disabled for background work, and the log is bounded to 256 collected lines before one truncation notice is added. Return integer, return string, error level, parameter string, and macro stack belong to the active execution state. They describe language execution and are not substitutes for coprocessor timing or UI messages.

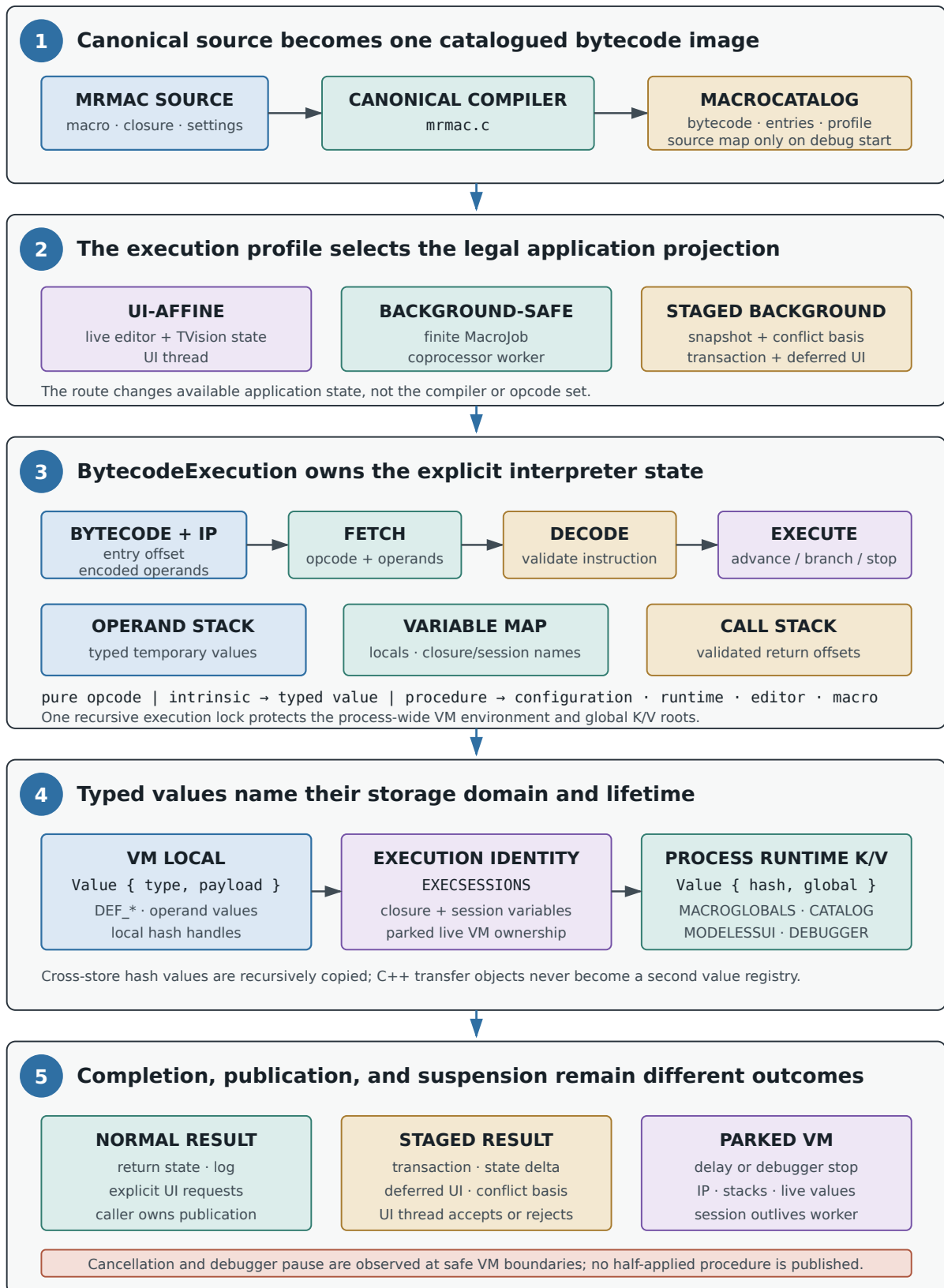
The coprocessor records the surrounding worker lifecycle: submission, assigned core, running interval, result readiness, acceptance, adoption, discard, and finish. An Execution Session records logical macro state and recent

outcome. The debugger records source-bound breakpoints and watches, while values shown at a stop are freshly projected from the live VM, closure/session backing store, and application globals. A debugger mutation is validated again by scope, type, and complete value path at the VM boundary and returns a new snapshot; the variable pane does not become writable runtime storage. Closure and session mutation publishes the VM value and its central K/V representation as one semantic transaction.

The global hash store has its own optional read/write-rate and hotspot instrumentation. That telemetry can show whether runtime K/V access has become a cost centre without placing counters inside macro-visible hashes or changing their values. Likewise, execution-profile flags describe possible behaviour found in bytecode; they do not record what a particular run happened to do.

INSIDE THE MRMAC VIRTUAL MACHINE

One bytecode model, explicit value lifetimes, and route-specific publication boundaries.



The VM owns executable state. TVision owns live views. Domain owners decide whether a result becomes visible.

Figure 9.1: The MRMAC virtual-machine boundary. Canonical bytecode enters through an execution-profile route, the VM interprets it with explicit stacks and typed values, and each result leaves through a route whose owner can validate it. VM-local values, Execution Session state, and process-wide K/V roots have deliberately different lifetimes.

Chapter 10

From a broken macro clock to Execution Sessions

The macro clock was a small experiment with a large lesson. Repeating macro work looked like concurrency, but it had no independent Execution Session. It therefore lacked identity, ownership, lifetime, cancellation, debuggability, and a reliable answer to the question of which work was still active.

An Execution Session now describes exactly one logical macro execution: session ID, owner, route, state, task, and, where applicable, closure belong together. The scheduler provides time and prevents uncontrolled overlap; it does not itself decide whether bytecode is UI-affine or suitable for the background. The runner selects the route: ordinary UI execution, foreground delay, background, or staged background.

The session is a logical lifetime, not a synonym for an operating-system thread. It may execute immediately, yield on `DELAY`, occupy one finite worker, wait for UI-thread result adoption, or remain parked in the debugger. The session ID survives these changes so status, cancellation, a scheduled consumer, and a debugger refer to the same work rather than reconstructing identity from a macro name.

User-visible session state lives below the central `EXECSESSIONS` K/V root. Active and pending sessions, recent terminal results, scheduler consumers and events, closure state, and variables of long-lived sessions are grouped by role below that root. C++ keeps only the mechanical objects that cannot be macro values: a callback function pointer, coprocessor task handle, or suspended VM. Those handles do not become a second session model.

The background routes also give the story a concrete hardware dimension. Each finite background or staged Execution Session submits a `MacroJob` in the macro lane. That work unit receives its own one-shot worker, monotone `workerOrdinal`, and stable modulo assignment to an allowed core. The lane classifies the job but does not serialize unrelated macro owners behind a single lane worker. UI-affine and foreground-delay sessions do not enter this path, and a session paused for UI adoption retains no worker.

Figure 10.1 on page 35 separates the routes and their lifetimes. The runner makes the routing decision; the scheduler is concerned with interval and overlap policy, while the coprocessor owns only the finite worker execution and result delivery.

The clock thereby became a shared foundation for recurring closures, modeless UI callbacks, execution status, and later the debugger. A scheduler that does not blindly place a second due execution next to the first is not a convenience-driven restriction. It protects retained UI state and external work from overlaps that cannot be explained afterwards.

10.1 Closures, timers, and explainable skips

A `$CLOSURE` unit is compiled by the canonical compiler like the rest of its macro file. Its `DEF_TICK` interval is scheduler metadata, not a new opcode and not a private timer loop. The runtime scheduler stores the consumer, closure identity, owner, interval, and next due time under `EXECSESSIONS`. On a due tick it asks the normal runner to create a session. It neither executes bytecode nor chooses a route.

The approved overrun policy is deliberately singular: skip. If the preceding session of that consumer is still active, the due tick records a `TickSkipped` event with the blocking session ID. Queueing a train of old timer events would make a modeless program look busy long after its relevant moment; parallel overlap would make retained closure and UI state ambiguous. A skipped tick is therefore observable but is not fabricated into an execution result, because no session ran.

Closure variables live below the closure's runtime identity and survive its individual callbacks. Ordinary finite `DEF_*` variables remain VM-local; explicit application globals remain in `MACROGLOBALS`. This separation is also what later gives the debugger meaningful scopes without inventing a file-global namespace.

MRMAC EXECUTION SESSION ROUTES

The logical session owns execution state; a finite background job owns only one worker lifetime.

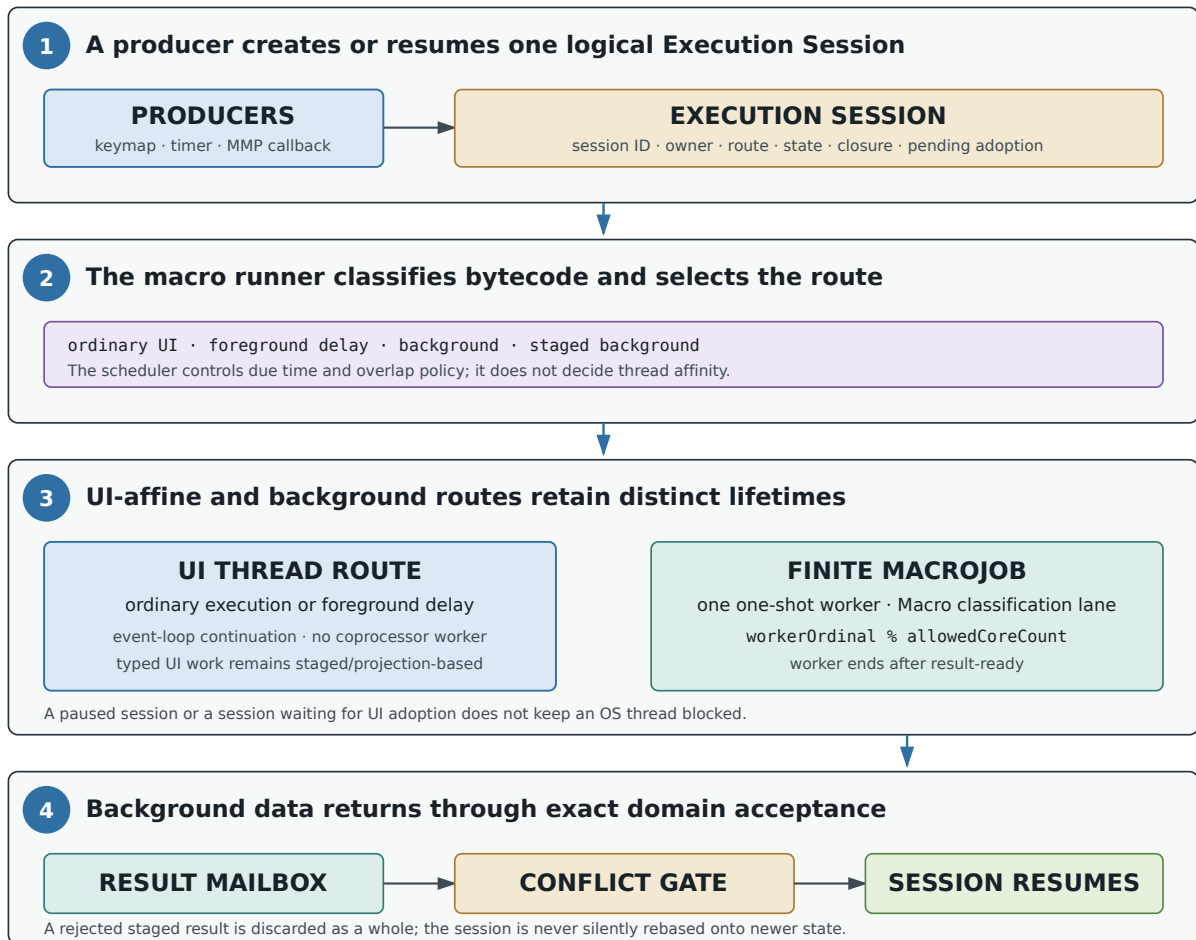


Figure 10.1: Execution-session routing. UI-affine and foreground-delay work remain on the UI thread. Each finite background or staged-background job receives an independently pinned one-shot worker in the Macro classification lane. The logical session may outlive that worker while its staged result waits for exact domain acceptance.

Chapter 11

Coprocessor, staging, and the UI thread

The VM must not alter a TVision view from a worker thread. For macro work that is suitable for the background, it therefore produces a staging result: text transactions, runtime state, and deferred UI commands are first collected as data. Only the UI thread evaluates the commit conditions against the current document and runtime state.

Before submission the runner captures one `ReadSnapshot`, its base version, cursor, selection and block state, relevant macro-visible runtime values, editor options, mark stack, file identity, and a conflict snapshot. The worker installs this package as a `BackgroundEditSession`. Editor operations write a staged transaction; supported UI requests become typed deferred commands. They do not find a live editor through a convenient global pointer.

An accepted commit changes the model first and then provides the deferred UI projection. A rejected commit is not partly repaired or rewritten for a newer version. This boundary explains why the UI pump does not “validate” in a general sense. It drains a bounded result mailbox; each result carries its own domain-specific acceptance check.

The conflict gate compares not only document version but the runtime state that would make the staged result meaningful. A match applies the Piece Table transaction and captured state, then queues deferred playback. A mismatch rejects the package as a whole. The debugger reuses this same path for staged bytecode; stopping a debug session discards the staged package, while normal completion still has to pass the ordinary conflict gate. Debugging is not a privileged route around transaction semantics.

This is what decoupling means technically in MR: not arbitrary threads being permitted to do things, but work whose result may become visible at an explicit boundary with an accountable owner.

Chapter 12

MMP: modeless programs without a second UI toolkit

MMP, the Modeless Macro Programs, were another point at which an apparently quick shortcut was rejected. Macros were not given raw TVision pointers, arbitrary paint callbacks, or a second event loop. They express typed, retained UI intent: a modeless window, native fields, action controls, selection data, a bounded log, or a canvas scene addressed by logical IDs.

The same boundary lets MMP state and the debugger meet in a Bento workspace without either view inventing execution truth. Debugger continuation, session identity, and value ownership remain the VM concerns described in Chapter 9.

This distinction makes MMP more than a collection of dialog procedures. A macro may build a dashboard, timer-driven monitor, interactive tool, or small application whose values outlive one callback. Yet the macro never owns a **TView**. TVision still owns live view objects, focus, Z order, event dispatch, drawing, and close semantics.

12.1 One value model, several projections

All retained MMP values have one authority: the central K/V tree rooted at **MODELESSUI**. Definition staging uses the **staging** subtree and its **currentDialog** child. Committed models use one child of **windows** per window ID. Text, boolean, bounded integer, progress, log, selection, tree and table state, canvas commands, scene generation, and live geometry are organised below those window models. A C++ dialog definition may temporarily transfer these values into native controls, but it is not an alternative registry.

The canvas follows the same rule. Its scene is retained K/V data made of bounded, palette-relative drawing roles. A canvas view reads that scene in its normal TVision **draw()** override. Commit requests a redraw of exactly that existing view; it does not flush a desktop, seize focus, or maintain a secret C++ scene cache. Ordinary controls use composed MMP constructs, while the canvas remains available for genuinely custom displays.

Figure 12.1 on page 38 shows the complete loop. The direction back from TVision is intentionally a typed execution request rather than an immediate call into a VM.

12.2 Callbacks are sessions, not event-handler re-entry

A modeless button, action menu, hotspot, or timer names a callback macro. Activating it does not execute bytecode inside the TVision event handler. The event becomes a request with the modeless window as execution owner, and the normal runner creates an Execution Session. Timer IDs exposed to MRMAC are logical IDs; scheduler consumer IDs remain internal mechanics.

That owner closes the lifecycle triangle. Closing an MMP window first removes its scheduled consumers, then cooperatively cancels its active sessions and removes its retained K/V subtree. A later result may not recreate expired closure or window state. This property is easy to miss in a clock demo and essential in a long-running personal extension environment.

MODELESS MACRO PROGRAM RUNTIME

MRMAC owns typed intent and retained values; TVision owns live views, events, focus and lifetime.

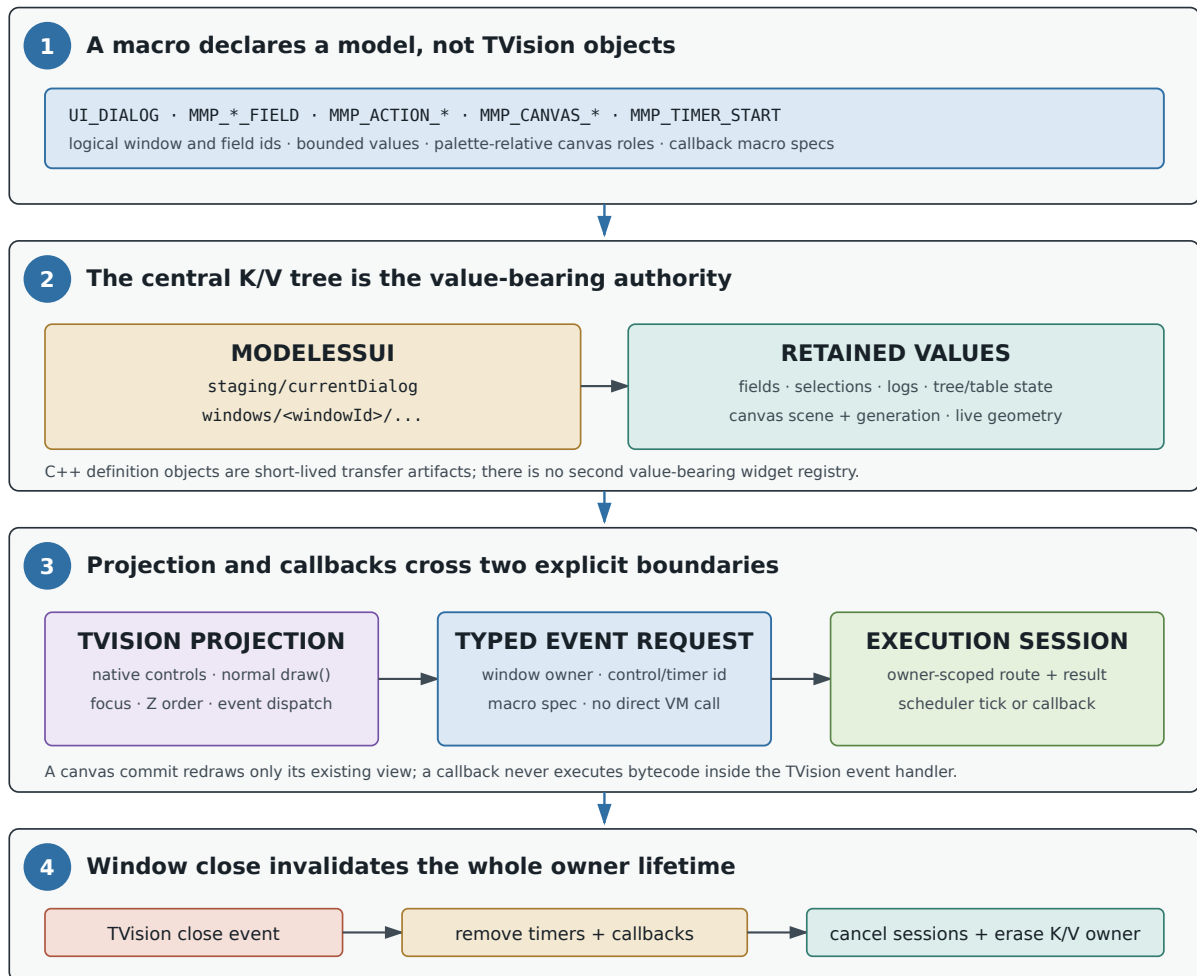


Figure 12.1: MMP runtime ownership. Typed macro declarations populate the authoritative **MODELESSUI** tree. TVision projects native controls and retained canvas scenes. Events and timers request owner-scoped Execution Sessions; closing the window removes scheduler consumers, requests cancellation, and erases the retained owner state.

Chapter 13

A debugger for the complete runtime

The integrated MRMAC debugger is a consequence of the runtime architecture, not a separate interpreter with a breakpoint veneer. It consumes Execution Sessions, the Macro Catalog, generated source maps, and central K/V values. Source, output, variables, and watches meet in a Bento workspace, but that workspace owns neither bytecode execution nor a writable copy of runtime state.

13.1 Source identity and source maps

A debug start resolves the macro through the same catalog as normal execution and invokes the canonical compiler with source-map collection enabled. Normal load does not pay that cost. Each mapping relates an instruction start to a source span, line, column, macro entry, and debuggable kind. Line breakpoints are a UI convenience: they bind to a real debuggable span rather than asking the UI to guess from bytecode.

The internal source identity is the normalised resolved path joined with the macro name by `^`. The UI renders only the macro name. Exactly one debugger Bento may own a source identity. Two views offering controls for the same parked VM would provide no useful capability and would make close, breakpoint, and mutation ownership ambiguous.

13.2 Natural execution routes and non-blocking control

The debugger adds stop policy—breakpoint, step, pause, cancellation, error, completion, or finite budget—to the normal runner. It does not add a debug lane. Background-safe code advances through finite jobs on the existing Macro lane. Staged code uses the captured input and ordinary commit path described in Chapter 11. Remaining UI-affine code advances in bounded event-pump slices. Even a run-to-cursor operation first yields control before substantive execution begins; its one-shot target survives that entry yield and is consumed only when the mapped target span is actually reached.

Figure 13.1 on page 40 makes the resource lifetime explicit. A worker executes only one finite Continue or Step action. At a breakpoint, pause, step stop, or budget boundary it returns a typed result. The VM may remain parked as a mechanical session handle, but no worker waits beside it.

Pause and Stop must also remain responsive while a worker currently owns the VM execution lock. Their controls therefore set cooperative request flags through a small mechanical control handle; they do not wait for the worker. The VM checks these requests at instruction boundaries. A C++ intrinsic, external I/O operation, or deferred playback application is a black box: stepping does not enter it and cancellation does not tear it in half.

Step Into, Step Over, and Step Out operate on source spans and call depth, including a loaded macro entered through `RUN_MACRO`. The call stack distinguishes ordinary calls from loaded child-macro frames. Where no source mapping exists, the debugger reports an opaque boundary instead of inventing a line.

13.3 Cold configuration and regenerated truth

Useful debugger configuration survives a workspace restart, but live execution does not. A debugger Bento persists its layout, macro and source identity, breakpoint source lines with enabled state and condition text, and watch expressions. It never persists a session ID, parked VM, route, current instruction, frame, value snapshot, output, generated source map, or bytecode offset.

DEBUGGER EXECUTION: ONE SESSION, NATURAL ROUTES

The debugger adds stop policy and inspection; it does not replace the macro runner, staging, or the UI pump.

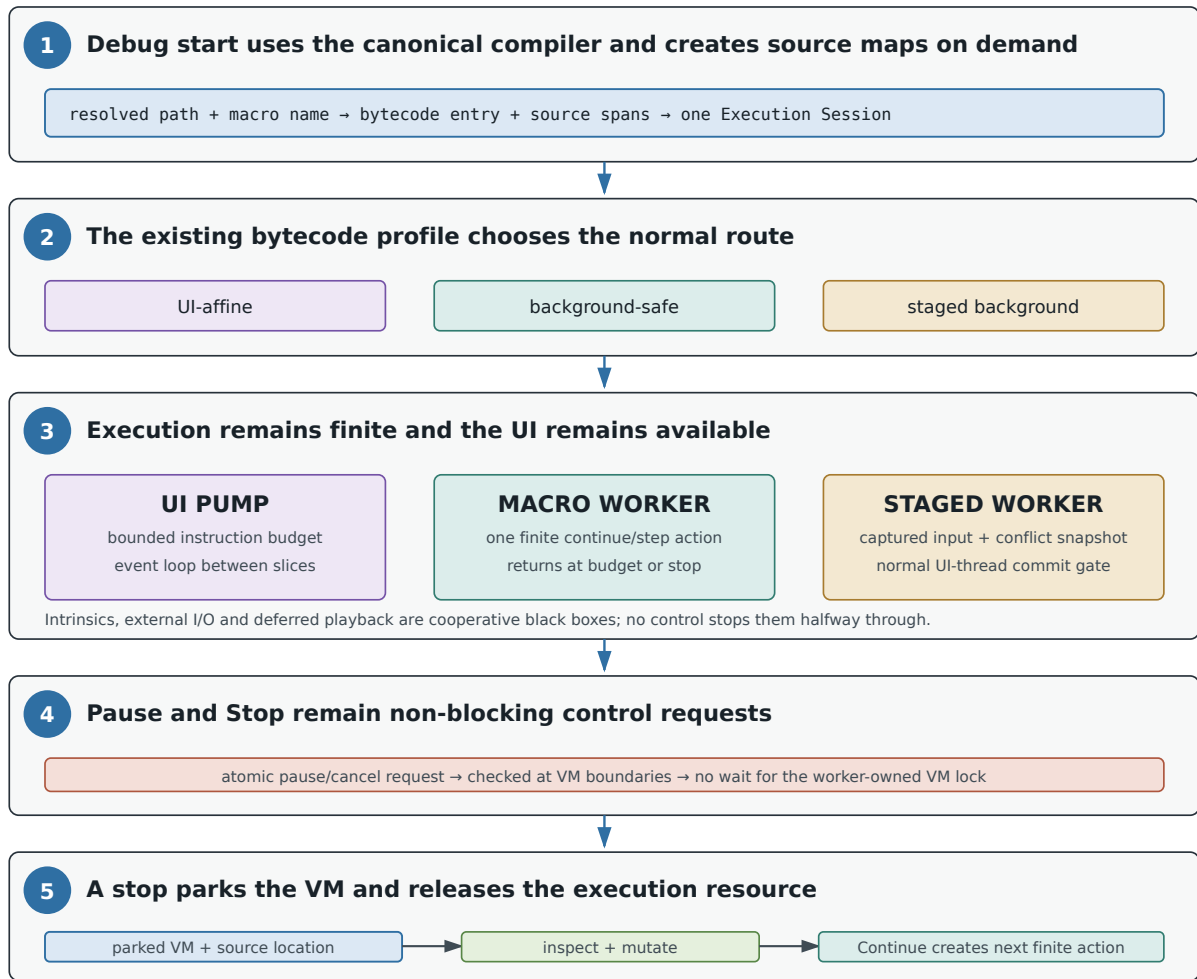


Figure 13.1: Debugger execution uses the macro’s natural route. UI-affine work is budgeted in the event pump; background and staged work use finite Macro workers. Cooperative pause and cancellation are lock-independent requests. A stopped VM is inspectable without retaining a worker.

Restore therefore produces a cold debugger. On a new start the source is compiled again, and breakpoint definitions are rebound against the new source map into **MACRODEBUGGER**. A breakpoint that no longer maps to an executable span and a watch that no longer compiles in restricted expression mode are dropped and logged during restore. They do not remain pending or unbound state. Unknown workspace options are likewise dropped and logged locally instead of making all known fields in the entry unrestoreable.

13.4 Scopes and complete mutable values

The Variables pane reports the scopes that actually exist: VM-local values, closure state, long-lived execution-session state, and explicit application globals. MRMAC has no file-global namespace. The apparent attraction of one is small because explicit globals already cross macro files, while its source-derived lifetime would add another ownership model.

Hashes and typed arrays are projected as full, indented trees. Hash keys and one-based array elements remain individually addressable through nested paths; cycles are marked rather than recursively expanded forever. There is no arbitrary snapshot cap. A container summary such as `hash{17 keys}` is useful orientation, but is not an inspection result.

Mutation occurs only while the VM is paused. The VM revalidates root name, scope, type, and every hash-key or array-index component before it changes anything. Scalars retain their type; hash entries may be added, renamed, or removed; array elements may be appended or removed; nested values use the same path. The appropriate local, closure, session, or global backing store is updated, and the UI receives a newly generated

DEBUGGER STATE: PERSIST DEFINITIONS, REBUILD EXECUTION

A cold debugger is useful after restore; a serialized live VM would be fiction.

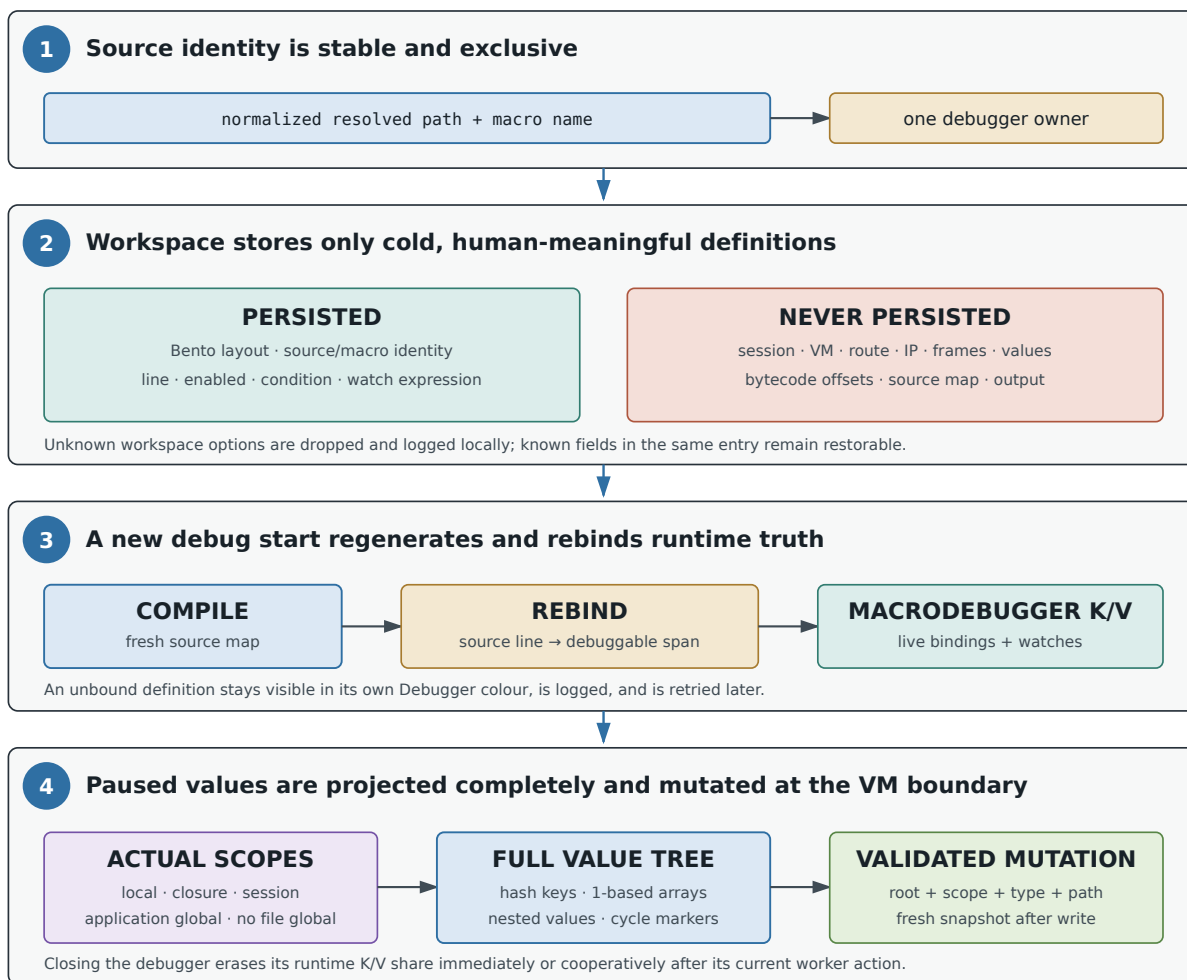


Figure 13.2: Debugger state boundaries. Workspace persistence keeps only cold definitions; a new debug start regenerates source maps and live K/V bindings. Paused values are projected from actual scopes as complete trees, and every mutation returns through a validated VM write boundary.

full snapshot. Closure and session writes prepare both representations before a no-fail publication under the VM execution mutex, so an error leaves both unchanged. Thus ergonomic editing does not turn the Variables pane into a second value authority.

Figure 13.2 on page 41 collects these persistence and value boundaries. Closing the debugger invalidates its live **MACRODEBUGGER** share. If a worker is still completing an atomic operation, cancellation and cleanup are recorded and performed cooperatively as that worker ends.

Watches use the canonical compiler in a restricted pure-expression mode. They may read paused state but cannot assign, call procedures, touch UI, or perform file and process work. Their expression and enabled state may be cold configuration; value and error are always derived anew. Together with the session timeline, scheduler skip events, call stack, variables, and log tail, this makes the debugger an observation surface for synchronous macros, MMP callbacks, recurring closures, and staged editor automation rather than only for the easiest straight-line case.

Part V

The editor as a personal environment

Chapter 14

settings.mrmac: canonical bootstrap and executable state

MR does not store its configuration as a JSON or YAML data heap. `settings.mrmac` is controlled bootstrap source in the language that belongs to the compiled application. The bootstrap reads the existing state, checks and normalises it, drops unknown or obsolete ends, adds current defaults, and writes canonical source with the current Build Epoch.

Figure 14.1 summarises the authority change from persisted source to the final VM apply.

The file is not then the live runtime owner. The VM executes the startup `MRSETUP` path; only the central in-memory settings state governs the running application. This “cutting off the old braid” approach avoids an ever-growing collection of compatibility lists. On loading, the past is translated into a current, readable, executable form.

There are explicitly named transitional tensions, including keymap staging while loading. It is not a hidden second authority, but a bounded exception because serialisation accesses the canonical keymap model. `WORKSPACE`, too, is a protected extension of persistence, not an unnoticed part of the canonical core settings contract.

SETTINGS BOOTSTRAP: PERSISTED SOURCE TO EDITOR UI

Bootstrap owns source normalization; the VM performs final apply; the runtime model is the sole live authority.

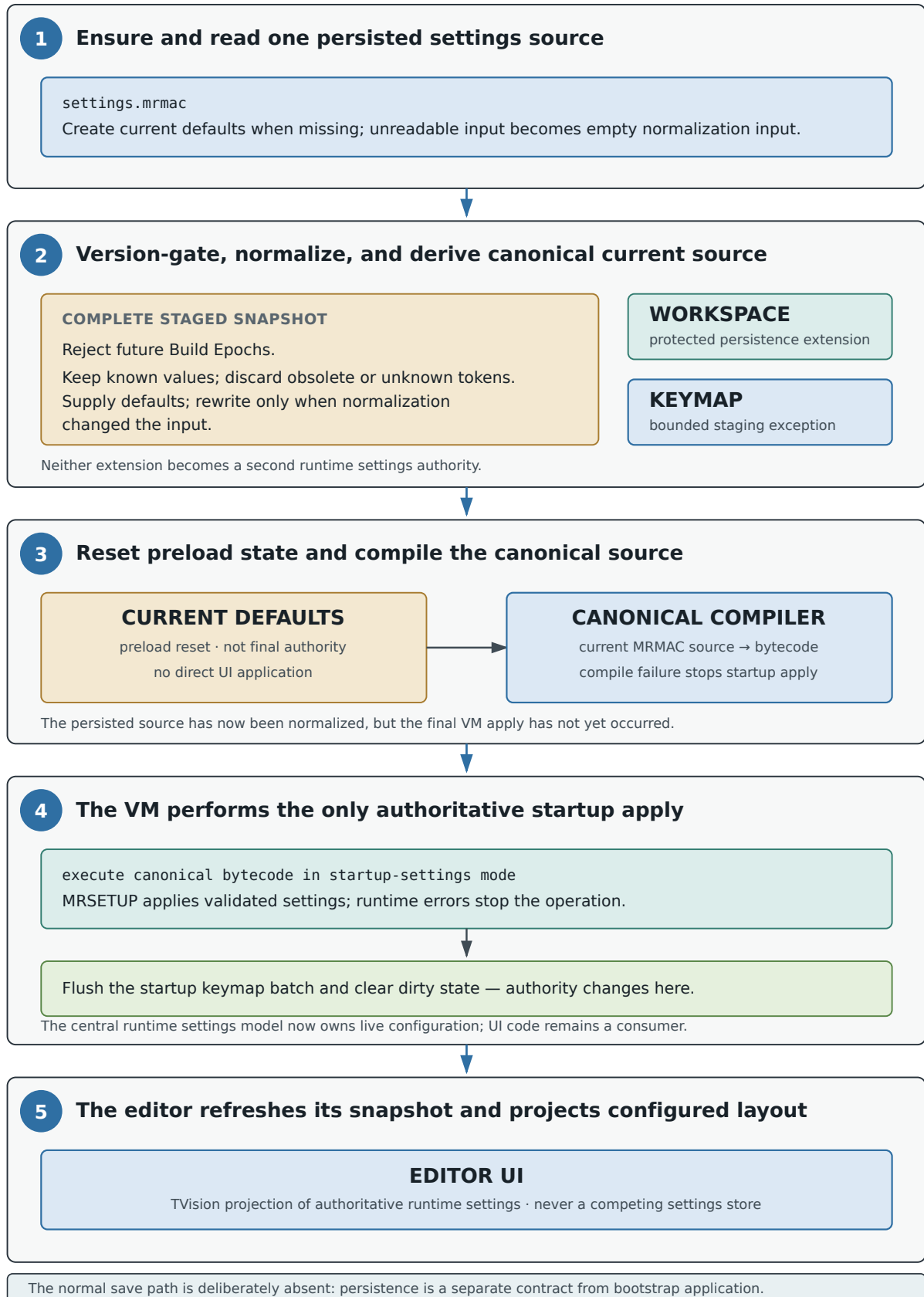


Figure 14.1: VM-centred settings bootstrap. A persisted source is checked, normalised, and made canonical before the VM performs the final startup MRSETUP apply. Only the successful VM route makes the central runtime settings model authoritative; the editor UI consumes that model.

Chapter 15

Keymaps as a programmable input language

The argument over efficient editor interaction does not end with a default. WordStar, Emacs, and Nano are not folklore in MR; they demonstrate that interaction is part of a personal way of working. A profile contains context-dependent canonical key tokens and bindings. The Key Manager makes this a usable editor feature rather than a configuration-file trick: it lets a user work with profiles and bindings, choose an action or macro target, and record a sequence live in canonical keymap syntax. Invalid sequences, collisions, and terminal/prefix conflicts are diagnosed before the profile becomes live. The resolver then builds the Keymap Trie as the runtime projection; the trie is not the feature's user interface, but the structure that makes its input grammar decidable.

A sequence is not a hard-coded single keystroke. It is stored as a token sequence and has no fixed length limit in the data model. The resolver may hold a prefix open, but it forbids terminal/prefix collisions. Thus there is no ambiguous waiting period in which a completed command might yet turn into a longer one.

What matters most is the target of a binding. It may be a fast built-in action or a file-qualified MRMAC macro. The user can therefore bind not just *Save* or *Cursor Left*, but their own complex, reviewable automation to their input language. This freedom is not unbounded: profile, context, token parser, trie, and the distinction between action and macro give it a form that can be understood.

Chapter 16

Profiles and automatic setup

File-extension profiles associate one or more selectors with effective editing settings, a window theme, and, where necessary, a compiler profile. A C++ project, a L^AT_EX document, and a shell script thereby receive not only a different colour, but an appropriate working environment. These profiles, too, are described through `settings.mrmac`, not hidden in a second, loose registry.

Compiler auto-setup extends the same idea. It finds known tools on `PATH`, probes their version and characteristics, and creates or completes profiles for toolchains that are actually present. That avoids empty configuration forms without pretending to understand every possible build system automatically. A profile remains visible and editable, and may include build hooks and MRMAC actions.

A compiler profile keeps execution policy visible: executable, toolchain, target, build flags, include, library, and runtime paths, a shell pre-build command, success and failure commands, result-dependent audio paths, and pre/post-build MRMAC hooks. These values remain ordinary central settings, not a registry owned by the compiler dialog.

Automatic setup has two bounded modes. With a selected executable it resolves and probes that compiler, then fills the supported profile fields from the probe. Without one it discovers the supported compiler families on `PATH` and merges named profiles without replacing existing IDs. The resulting profile is still editable and serialises through the canonical settings route.

At execution time the pre-build MRMAC hook runs before submission. The external shell task exports one explicit build context, executes the shell pre-command, conditionally runs the compiler, and dispatches the success or failure command while preserving the build status. UI-thread result adoption then performs the matching audio notification and post-build MRMAC hook. Thus shell policy remains inside the finite external task, while VM mutation and visible adoption remain on their authorised side of the boundary.

Part VI

Syntax as a shared core

Chapter 17

From an external generator to an integrated syntax model

The first attempt to recognise MRMAC sources used Lex and Bison. That approach proved outdated and did not provide a viable foundation for an editor whose language knowledge must serve colouring, folding, and indentation at the same time. The version history preserves at least the trace of removing the Flex/Bison material; the engineering lesson matters more than the archive: a parser generator does not itself solve the problem of an integrated, incrementally usable editor model.

The present core combines classification by path, extension, and content probe with shared line state and token runs, plus concrete rules for individual languages. It deliberately does not claim to be a complete universal grammar. Each language supplies its syntax state and token runs through the same contract as the others; folding, outline, colouring, and indentation decisions can reuse those results. MRMAC is consequently not a foreign C component beside the editor, but a language in its own syntax model.

The current tree has 24 language modes, including Plain Text, not 384. Automatic recognition combines file names, extensions, and a bounded content probe. The number is smaller than the perceived breadth of the editor, but it is the honest number. A later Tree-sitter trace was also removed from the repository; the history proves the removal, but does not establish its exact reason without further sources. This manual does not invent one afterwards.

Chapter 18

Trainers: the workbench for language features

The trainers are not machine learning. They are a workbench on which language rules become visible, repeatable, and discussable against real examples. The fold trainer automatically classifies an input when needed and emits its fold structure as numbered ASCII. The outline trainer uses the same fold scan to produce the outline.

The indent trainer goes further. It creates a real `MRFileEditor`, varies language, indentation style, UI style, and tab treatment, and simulates Enter and Smart Undent. Its strict pass/fail expectations therefore exercise production paths rather than an independently invented test semantics.

The trainers do not prove the entire visible behaviour under load; they are no replacement for UI or coprocessor checks. They do secure the exact semantic question that they show: which structure, outline element, and indentation decision follows from this text? That is valuable testability because it is made of the same language as the editor feature.

Part VII

Spaces and the Unix world

Chapter 19

Virtual desktops and resumable workspaces

A terminal editor need not be reduced to a single document window. MR assigns every desktop-capable window to a virtual desktop. When changing desktops, windows on the other desktop are hidden, not destroyed; focus, geometry, and state remain intact. Windows can be moved, minimised, tiled, and cascaded without those desktop operations needing to know the domain nature of an editor, a Bento view, or a modeless macro window.

A workspace makes this spatial working state recoverable. It stores, among other things, URLs, window geometry, cursor positions, virtual desktop, minimised and restore state, and Bento snapshots for suitable windows. Autoload and autosave do not turn it into an all-powerful persistence layer; they make it a deliberately bounded continuation of personal work.

Runtime execution identity is intentionally absent. Workspace restore does not serialise or recreate worker ordinals, task IDs, OS threads, core assignments, packet reservations, or warmed ledgers. It restores windows, panes, widget masks, and domain state; constructors and restored demand then create new execution owners and fresh packets. Background windows continue their work after restore without inheriting a stale worker lifetime.

The result is a TUI workspace in which many windows need not form an unmanageable parade. They can live on virtual desktops and reappear in a workspace exactly where work was interrupted.

Chapter 20

Git as a file-scoped external view

Git support does not introduce a repository model beside Git itself. When a named editor gains focus or its file state changes, the window submits a finite `Io` task owned by that editor's buffer identity. The probe asks Git for porcelain status of exactly that path. Exit status distinguishes clean, changed-or-untracked, and unavailable; command output is not retained as application state.

The result carries task ID, file path, and a monotonically changing window generation. Adoption succeeds only if the same window still owns the same path and generation. Rename, reload, save, focus-driven invalidation, or close therefore prevents a late status result from painting a marker for an obsolete file. The frame's Git marker is a projection of the adopted boolean, not a cached repository index.

The explicit Git Changes command is a second finite external route. A dirty editor must first save successfully. The command then creates a communication window and asks Git for repository root, branch, short status, and a file-scoped diff; an untracked file uses an empty source for comparison. Output travels through the ordinary external-command worker and UI adoption path. There are deliberately no staging, commit, checkout, or branch-state objects in MR: the feature provides file context without becoming a second Git client.

Chapter 21

MFS and Acquire: Unix output as working material

Multi-file search and multi-file replace combine recursive file work with PCRE2 and can be restricted to the workspace. This is not merely a search box with a different file dialog. It is a bridge between the open workspace and the file tree.

MFS remains modal so that editor snapshots and replacement offsets cannot silently become stale through concurrent editing. Directory traversal, chunked file I/O, snapshot materialisation, and PCRE2 matching nevertheless run in a finite dialog-owned coprocessor task. The dialog reports collection and x/y search progress. Its Search button becomes Cancel while work is running. Cancellation is checked during traversal, between I/O chunks, between matches, and through PCRE2 automatic callouts inside a long individual match. Only a complete, still version-valid session reaches the result dialog. The result model owns one entry per candidate file and its matches, so Load opens the selected file while Load All opens each distinct result file rather than reloading once per hit. Replace All remains a UI-thread mutation phase, but performs one replacement per dialog event and reports x/y progress. Matches are applied from the end of each file so that earlier offsets remain valid without rescanning the complete file after every edit. Cancel pauses the mutation and offers Continue, Abort, or Revert. Abort retains the completed prefix as individual Undo records. Revert restores one pre-operation snapshot per touched file and removes only the Undo suffix owned by that Replace All run; it does not replay thousands of Undo commands or construct a corresponding Redo stack.

Acquire follows another, particularly terminal-native idea. MR runs a shell command out of sight, reads its standard output, and interprets each line as a candidate readable path. Candidates are resolved and deduplicated; the user may open or load selected files or every discovered file. The process group runs in a finite I/O worker and stdout is adopted incrementally. The list shows per-entry ordinals and a total. Closing the dialog cancels the process group, waits for its bounded TERM/KILL teardown, and discards unadopted output. Loading all files is sliced at file boundaries so that Close can stop the remaining queue. During that batch the central message-line Static Mode projects completed/total progress and gives Escape to the cancellation route; already loaded files remain valid. Acquire is therefore not the uncritical insertion of arbitrary pipe bytes into a buffer. It turns Unix tools into an understandable file selection.

Part VIII

Decisions that left traces

Chapter 22

The Hex restart and the LSP train

The first Hex integration was discarded. Its lesson was specific: a Hex view does not belong as a special branch in a generic BentoBox; it needs neither an exception to the normal pane lifetime nor a self-drawn frame. Restarting it as `MRBentoHexEditor`, with six ordinary pane windows, shared byte-cursor logic, and the same document basis, was not cosmetic refactoring. It restored clear responsibility. Moving string recognition, Inspector conversion, and numeric formatting from `draw()` into six pane-owned immutable projections completed that responsibility: the Hex restart now separates the shared byte model, derived worker data, and TVision projection in the same way as the rest of the editor.

The LSP train was more painful. Two attempts consumed weeks; the architecture could not be shaped into a viable editor feature. The history ultimately records the removal of the LSP material in a large rollback. This is not a claim that LSP is inherently bad. It is the specific conclusion that an external IDE model could not be integrated with adequate clarity and benefit into this project's existing runtime, UI, and working model.

Both stories belong in the manual because discarding work can be a technical ability. A system remains coherent not by saving every line once written, but by being able to remove the wrong boundaries again.

Chapter 23

Measured limits, not wishful thinking

The subsystem evidence described in Chapters 4 and 9, together with trainers and the live log, makes technical claims open to challenge. This chapter records measured outcomes and limits rather than defining those telemetry contracts a second time.

The correction train also preserved quantitative counterexamples. Repeated random line addressing made the original enwik9 display-width pass run for more than six minutes; one initial address plus linear `nextLine` reduced the 13,147,026-line generation to 5.434–6.384 seconds including debugger stops. Syntax packet density fell from 7.00 to 4.07 packets per generation, and display-width result churn for the measured version sequence fell by 92.8 percent. HexBento’s combined navigation bursts settled in 0.13–0.17 seconds, and an identical 954-MiB File Compare remained responsive and closed without residual workers.

These numbers do not make a high absolute one-shot count a defect by itself. The relevant conditions are bounded packets, exact owners, strict modulo core assignment, no lane-wide serialization, bounded UI adoption, and complete return to zero after load or close. An exceptional single long line remains sequential within its width packet; syntax and folding remain ordered at their validation fronts; a genuinely blocking external stream may retain its own worker. Those are measured and named limits, not concealed scheduler queues.

The build belongs to this story as well. It orchestrates the TVision upstream, the MRMAC compiler, training programs, regression checks, and manuals. A clean rebuild is not a ritual; it tests whether the delivery form claimed by the repository can actually be produced from its source. The manually maintained Bento header dependencies are not yet complete, so a header-layout change is trusted only after `make clean all CXX=clang++`; an incremental link may otherwise combine ABI-incompatible objects.

Outlook

The technical centre of MR is not one individual trick. It is the repeated application of the same attitude: do not copy original data unnecessarily, decouple derived work from interaction, give truth one unambiguous owner, and place extensibility under the same boundaries as built-in code. This is why a gigabyte document can become useful immediately, a Braille minimap can exist in a console, a user can program an input language, and a deferred macro can remain debuggable.